Cummulative hierarchies of universes and their equivalence in dependent type theory

RAPHAËL STERBAC, ENS Paris-Saclay — Université Paris-Saclay, France
THIERRY COQUAND, University of Gothenburg, Sweden

In this paper, we present a modular and algorithmic proof for the equivalence of different presentations of type theory, notably relating Russell and Tarski presentations for a cumulative hierarchy of universes. More generally, we relate generalised algebraic and pure type system formulations. Crucial lemmas are injectivity of type formers, and "no-confusion", which we get by proving normalization for the Tarski-style theory via an adaptation of the gluing technique. Our work can be seen as an extension of initiality results to more informal presentations of type theory.

1 Introduction

1.1 Motivation

In type theory, universes

There are two different ways to formulate the notion of universe in type theory. Martin-Löf's first versions [15, 16] were formulated à la Russell, where a universe is seen as a type of types. Later, an alternative version à la Tarski [17], where a universe is seen as a type of codes along with a decoding function, was also introduced. Usually, Russell universes are regarded as an informal notation of the latter, as [14] suggested.

When presenting Type Theory as a Generalised Algebraic Theory, Tarski universes are mandatory, and one can show that the system à la Tarski presents the initial model of such a theory. This initial model has been constructed explictely in [4], following the method presented in [18] and formalised in [9]. However, there are as many initiality theorems as there are presentations of type theory, and if people often assume that they extend easily to the more informal syntax used in practice, this comes from misconceptions as they depend on subtle details of the presentation. To solve this issue, Voevodsky proposed the *Initiality Conjecture project*, which aimed at developing rigorous metatheory for large class of theories. A first step at solving this goal was achieved by Streicher [18], proving initiality for a fully annotated Calculus of Constructions, and then proving its equivalence with the more common, unannotated, Pure Type System formulation.

Generalised Algebraic Theories provide a framework to describe type theories and their model in a uniform way, and to prove initiality in a easy way [4], thus solving the first part of the problem. However, they are a fully explicit and annotated presentation of type theory, which is inconvenient to work in, and it would be essential to extend Streicher equivalence results to relate Generalised Algebraic and Pure Type System presentations, where the latter is presented with Russell universes and unannotated syntax.

We shall here show that Streicher's technique can be extended to show the equivalence of the Russell and Tarski formulations of a hierarchy of universes.

1.2 Previous work

Some recent works [2, 19] show the equivalence between these two formulations for the Calculus of Constructions with β as an external reduction. However, these arguments do not apply to formulations of type theory with judgemental equality and do not extend to η-equivalence.

Authors' Contact Information: Raphaël Sterbac, raphael.sterbac@ens-paris-saclay.fr, ENS Paris-Saclay — Université Paris-Saclay, Gif-sur-Yvette, France; Thierry Coquand, Thierry.Coquand@cse.gu.se, University of Gothenburg, Gothenburg, Sweden.

The work of [2] provides the structural equations necessary for this equivalence, by achieving what he calls *full reflection*.

In the papers [2, 14, 19], they consider cumulativity with a structural sub-typing judgement $A \leq B$ which can lead to discrepancy between Russell and Tarski universes in a more general setting, as pointed out in [14]. In fact, when adding inductive types such as a type of lists $\mathsf{list}(.)$, the sub-typing relation propagates along this new constructor: $A \leq B$ implies $\mathsf{list}(A) \leq \mathsf{list}(B)$. Concretely, this means that we get $\mathsf{list}(\mathsf{U}_0) \leq \mathsf{list}(\mathsf{U}_1)$, already breaking canonicity as we thus have $\mathsf{nil}_{\mathsf{U}_0} : \mathsf{list}(\mathsf{U}_1)$ with $\mathsf{nil}_{\mathsf{U}_0} \neq \mathsf{nil}_{\mathsf{U}_1}$.

Therefore, we believe that such a sub-typing relation is not desirable for our theory. This is also true for Π-types, the relation $B \leq B'$ implying $\Pi(A, B) \leq \Pi(A, B')$ is not true in the Set model, or at least unclear depending on the encoding of application in the set theory. Yet, there are some cases where structural sub-typing can be useful, as described in the paper [12]. The recent work [1] also provides a general framework which would provide the necessary equations to solve the issues mentioned above.

1.3 Contributions

In this paper we show the equivalence for this case by a method due to Streicher [18] that seems more modular and transparent than the one in [2, 19]. Therefore, we present a proof of equivalence between Russell and Tarski universes for type theory with dependent products, cumulative universes and judgmental $\beta\eta$ equality. The main argument is as follows: there is a natural forgetful map from the Tarski version to the Russell version, and one can show that any Russell judgement can be lifted in a unique way to a Tarski judgement.

Our proof adapts to a theory with universe polymorphism and internal universe levels such as in [3]. We rely on basic properties of Type Theory like no confusion and injectivity of type constructors, which seem to be strongly related to conversion [13]. We get those properties by showing normalisation of the Tarski theory, by an adaptation of the gluing proof from [7] Then, we show the proper equivalence result by adapting the method of [18].

The proof is elementary and provides an algorithm to translate from one presentation to the other. We formalised the proof of equivalence in Rocq, taking the results given by normalization as axioms. The development is available here.

2 Models and syntax

In this section, we define the theory with Tarski-style universes as a generalized algebraic theory (GAT), which both presents the syntax in an algebraic way, and provides a canonical notion of model in categories with families (CwF). Those were originally described in [5], and extended with dependent products and universes by [6].

Our Generalised Algebraic Theory of Categories with families has four sorts: contexts, substitutions, types, and elements:

$$\vdash \mathsf{ctx}$$
$$\Delta, \Gamma : \mathsf{ctx} \vdash \Delta \longrightarrow \Gamma$$
$$\Gamma : \mathsf{ctx} \vdash \mathsf{Type}(\Gamma)$$
$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma) \vdash \mathsf{Elem}(\Gamma, A)$$

It also has operations making ctx a category (of contexts) which acts on types and terms.

Definition 2.1 (Substitutions). It has a composition and identity operation:

$$\Gamma : \mathsf{ctx} \vdash \mathsf{id}_\Gamma : \Gamma \to \Gamma$$
$$\Theta, \Delta, \Gamma : \mathsf{ctx}, \sigma : \Delta \to \Gamma, \delta : \Theta \to \Delta \vdash \sigma\delta : \Theta \to \Gamma$$

which satisfies the following equations (we ommit the contexts for the sake of simplicity) :

$$\sigma\,\mathsf{id}_\Gamma = \mathsf{id}_\Gamma\,\sigma = \sigma \qquad\qquad (\theta\sigma)\delta = \theta(\sigma\delta)$$

We also require a terminal object in this category, written 1 and corresponding to the empty context:

$$\vdash 1 : \mathsf{ctx}$$
$$\Gamma \vdash ()_\Gamma : \Gamma \to 1$$

Subsitutions are acting on types and elements, in the sense of the following definition.

Definition 2.2 (Action of substitutions on types and terms). Our GAT has the following operations:

$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma), \sigma : \Delta \to \Gamma \vdash A\sigma : \mathsf{Type}(\Gamma)$$
$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma), \sigma : \Delta \to \Gamma, a : \mathsf{Elem}(\Gamma, A) \vdash a\sigma : \mathsf{Elem}(\Delta, A\sigma)$$

Satisfying the following equations:

$$A\,\mathsf{id} = A \qquad (A\sigma)\delta = A(\sigma\delta) \qquad a\,\mathsf{id} = a \qquad (a\sigma)\delta = a(\sigma\delta)$$

We also need context extension, in the sense of the following.

Definition 2.3 (Projections). Our GAT has the following operations:

$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma) \vdash \Gamma.A : \mathsf{ctx}$$
$$\Gamma, \Delta : \mathsf{ctx}, A : \mathsf{Type}(\Gamma), \sigma : \Delta \to \Gamma, a : \mathsf{Elem}(\Gamma, A) \vdash (\sigma, a) : \Delta \to \Gamma.A$$
$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma) \vdash \mathsf{p}_{\Gamma,A} : \Gamma.A \to \Gamma$$
$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma) \vdash \mathsf{q}_{\Gamma,A} : \mathsf{Elem}(\Gamma.A, A\mathsf{p}_{\Gamma,A})$$

Satisfying the following equations:

$$\mathsf{p}(\sigma, a) = \sigma \qquad \mathsf{q}(\sigma, a) = a \qquad (\sigma, a)\delta = (\sigma\delta, a\delta) \qquad (\mathsf{p}, \mathsf{q}) = \mathsf{id}$$

All of this defines an *explicit* substitution calculus, from which we can recover the usual presentation of substition, as defined in the following.

Definition 2.4 (Usual syntax for substitutions). If $a : \mathsf{Elem}(\Gamma, A)$, we write $[a] = (\mathsf{id}, a) : \Gamma \to \Gamma.A$. We also define $\sigma^+ = (\sigma\mathsf{p}, \mathsf{q}) : \Delta.A\sigma \to \Gamma.A$ for a substitution $\sigma : \Delta \to \Gamma$ and a type $A : \mathsf{Type}(\Gamma)$. This extends our substitution σ to the extended context $\Gamma.A$.

Now, we can add logical structure by requiring type connectives in our theory, starting with dependent products.

Definition 2.5 (Dependent product). We require the following operations, corresponding to the dependent product with lambda abstraction and application:

$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma), B : \mathsf{Type}(\Gamma.A) \vdash \Pi(A, B) : \mathsf{Type}(\Gamma)$$
$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma), B : \mathsf{Type}(\Gamma.A), b : \mathsf{Elem}(\Gamma.A, B) \vdash \lambda(A, B, b) : \mathsf{Elem}(\Gamma, \Pi(A, B))$$
$$\Gamma : \mathsf{ctx}, A : \mathsf{Type}(\Gamma), B : \mathsf{Type}(\Gamma.A), f : \mathsf{Elem}(\Gamma, \Pi(A, B)), a : \mathsf{Elem}(\Gamma, A) \vdash \mathsf{app}(A, B, f, a) : \mathsf{Elem}(\Gamma, B[a])$$

This type connective must be functorial :

$$\Pi(A, B)\sigma = \Pi(A\sigma, B\sigma^+)$$

Those should be functorial and satisfy the β and η rules, given by the following equations.

$$\Pi(A, B)\sigma = \Pi(A\sigma, B\sigma^+)$$
$$\lambda(A, B, b)\sigma = \lambda(A\sigma, B\sigma^+, b\sigma^+)$$
$$\mathsf{app}(A, B, (\lambda(A, B, b), a)) = b[a]$$

$$c = \lambda(A, B, \mathsf{app}(A\mathsf{p}, B\mathsf{p}^+, c\mathsf{p}, \mathsf{q}))$$
$$\mathsf{app}(A, B, c, a)\sigma = \mathsf{app}(A\sigma, B\sigma^+, c\sigma, a\sigma)$$

We now add cumulative universes to our theory, by requiring a countable hierarchy of universes $\mathsf{U}_0, \mathsf{U}_1, \ldots$, along with explicit codes, lifting and decoding operations. This presentation of universes is usually referred to as Tarski-Style universes. This explicit system is the only known (generalised) algebraic presentation for type theory with cumulative universes.

Definition 2.6 (À la Tarski universes). In our Generalised Algebraic Theory, we add a countable list of universes U_n with a decoding operation $T_n(a)$ which takes a code (i.e. an element of a universe) $a : \mathsf{Elem}(\Gamma, \mathsf{U}_n)$ and produces a type.

We also close each universe under all the type connectives. Namely, we require $\Pi^n\ A\ B$ for $n \geq 0$, which are codes for Π-types in U_n, alongside with U_n^k which are codes for U_k in U_n (with $k < n$).

Cumulativity is encoded by a lifting operation t_n^k that takes a code in U_k, and produces a code in U_n for $k < n$. To summarize, we need all the following operations:

$$\Gamma : \mathsf{ctx} \vdash (\mathsf{U}_l)_\Gamma : \mathsf{Type}(\Gamma)$$
$$\Gamma : \mathsf{ctx} \vdash (\mathsf{U}_l^m)_\Gamma : \mathsf{Elem}(\Gamma, (\mathsf{U}_m)_\Gamma)$$
$$\Gamma : \mathsf{ctx}, a : \mathsf{Elem}(\Gamma, (\mathsf{U}_l)_\Gamma) \vdash t_l^m(a) : \mathsf{Elem}(\Gamma, (\mathsf{U}_m)_\Gamma)$$
$$\Gamma : \mathsf{ctx}, a : \mathsf{Elem}(\Gamma, (\mathsf{U}_l)_\Gamma) \vdash T_l(a) : \mathsf{Type}(\Gamma)$$
$$\Gamma : \mathsf{ctx}, a : \mathsf{Elem}(\Gamma, (\mathsf{U}_l)_\Gamma), b : \mathsf{Elem}(\Gamma.T_l(a), (\mathsf{U}_l)_\Gamma) \vdash \Pi^l(a, b) : \mathsf{Elem}(\Gamma, (\mathsf{U}_l)_\Gamma)$$

The decoding and lifting maps must be strict and natural:

$$T_l(t_n^l(X)) = T_n(X) \text{ for } l \leq n$$
$$T_n(\Pi^n\ X\ Y) = \Pi(T_n(X), T_n(Y)) \text{ for } n \geq 0$$
$$T_n(\mathsf{U}_m^n) = \mathsf{U}_m \text{ for } m < n$$

$$t_n^p(t_m^n(X)) = t_m^p(X) \text{ for } m < n < p$$
$$t_n^p(\Pi^n\ X\ Y) = \Pi^p\ t_n^p(X)\ t_n^p(Y) \text{ for } n < p$$
$$t_n^p(\mathsf{U}_m^n) = \mathsf{U}_m^p \text{ for } m < n < p$$

It is important to note that those equations have been carefully chosen so that this presentation is equivalent to the *implicit* Russell universes. In fact, those naturality and functoriality equations are already well-known and already appear in the work of [2, 19], where they are described as requiring "*full reflection*".

All of this defines a Generalised Alebraic Theory of Categories with Families like with dependent products and a hierarchy of universes. We can then construct the initial model of this theory, which correspond to the syntax of the corresponding type theory. This initial system will be called $\mathcal{T}_G$, and can be presented with more usual inference rules, while keeping substitution explicit. We also consider the identical system $\mathcal{T}_T$, but with implicit substitutions.

The main goal of this paper is to relate $\mathcal{T}_T$ to the implicit *"à la Russell"* formulation of cumulative universes, which adds universes directly as $\mathsf{U}_m : \mathsf{U}_n$ for $m < n$, and enables to deduce $A : \mathsf{U}_m$ from $A : \mathsf{U}_n$ if $n < m$, with no codes or liftings involved. In this presentation, there is no syntactical distinction between terms and types.

Such a system cannot be formulated as a GAT, as in this setting we can't have one universe belonging to different sorts at the same time.

The system with Russell-style universe will be called $\mathcal{T}_R$, and is also described in detail in appendix with inference rules.

Finally, we also consider a system $\mathcal{T}_P$, which removes redundant type annotations in application and abstraction from $\mathcal{T}_R$. Thus, $\mathsf{app}(A, B, c, a)$ and $\lambda(A, B, b)$ becomes $c\ a$ and $\lambda(A, b)$. This system is closer to the usual syntax of type theory, inherited from the lambda calculus, and often referred to as the "pure type system" presentation (PTS). This system is also described in appendix.

In the next sections, we will prove the equivalence of those systems. The links between them are summarized in the following figure, where the strong arrows are the known forgetful maps, and the dotted arrows are the sections we will be constructing.

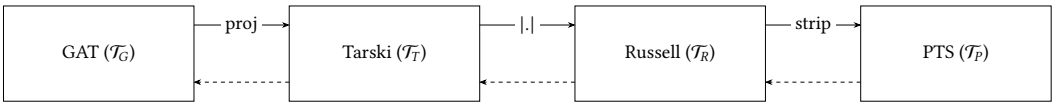

Fig. 1. The hierarchy of our systems (from left to right)

3 Normalisation

In this section, we show that the system on top of our hierarchy ($\mathcal{T}_G$) normalises. To prove normalisation for this theory, we apply the gluing technique presented in [7], which can be seen as a proof-relevant formulation of the usual Tait's normalisation proof. This proof is adapted here to a system with Tarski-style universes, by only presenting the corresponding construction for universes.

Let M be an arbitrary model. In the following, we write Type_M and Elem_M for its types and elements.

3.1 Presheaf models

In the follwoing, we will construct and work with a presheaf model of type theory, which is a category with families constructed as the category of Set-valued functors over a special base category. The latter will be the category of telescopes.

Definition 3.1 (Telescopes). A telescope X is a finite list of types $A_0, A_1, ..., A_{n-1}$ with A_0 in $\mathsf{Type}_M()$, A_1 in $\mathsf{Type}_M(().A_0)$ and so on. We define its interpretation $\langle X \rangle = ().A_0. ... .A_{n-1}$ which is a context of M.

Maps $\alpha : Y \to_S X$ between two telescopes are syntactic objects, constructed inductively in the natural way, with syntactic versions of $()$, p and $.^+$. We also have an interpretation $\langle \alpha \rangle$ of a map as a substitution, and hence such a map α can be seen as a witness that the telescope Y extends X.

We also define an identity map $\mathsf{id}_S : X \to_S X$, a composition operation $\alpha\beta$, and a syntactic projection p_S such that $\langle \mathsf{p}_S \rangle = \mathsf{p}$.

This defines the category C of telescopes on which we define a presheaf category $\widehat{C}$ which objects are functors $F : C^{\mathrm{op}} \to \mathsf{Set}$.

Such a presheaf category defines a model of type theory in the sense of category with families, as shown in [10]. In this model we assume to have a cumulative sequence of universes $\mathcal{U}_n$.

We now introduce the sets $\mathsf{Elem}_N(X, A)$ of normal terms, $\mathsf{Type}_N(X)$ of normal types, and $\mathsf{Neut}(X, A)$ of neutral terms, along with their interpretation $\langle . \rangle$. Those definitions are analogous to the proof by reducibility candidates.

Definition 3.2 (Neutral and Normal terms). We define $\mathsf{Elem}_N(X, A)$, $\mathsf{Type}_N(X)$ and $\mathsf{Neut}(X, A)$ for X a telescope and A in $\mathsf{Type}_M(\langle X\rangle)$, simultaneously by induction, along with their interpretation $\langle . \rangle$:

$$
\begin{aligned}
\mathsf{U}_n &\in \mathsf{Type}_N(X) \text{ for } n \geq 0.\\
\Pi\, K\, L &\in \mathsf{Type}_N(X) \text{ if } K \in \mathsf{Type}_N(X) \text{ and } L \in \mathsf{Type}_N(X.\langle K\rangle).\\
T_n k &\in \mathsf{Type}_N(X) \text{ if } k \in \mathsf{Neut}(X, \mathsf{U}_n).\\
v_0 &\in \mathsf{Neut}(X.A, A\mathsf{p}), \text{ and } v_{n+1} \in \mathsf{Neut}(X.A, B\mathsf{p}) \text{ if } v_n \in \mathsf{Neut}(X, B).\\
\mathsf{app}\, K\, L\, k\, t &\in \mathsf{Neut}(X, \langle L\rangle[\langle t\rangle]) \text{ if } K \in \mathsf{Type}_N(X),\ L \in \mathsf{Type}_N(X.\langle K\rangle),\ k \in \mathsf{Neut}(X, \langle \Pi\, K\, L\rangle),\\
&\quad \text{and } t \in \mathsf{Elem}_N(X, \langle K\rangle).\\
\lambda\, K\, L\, t &\in \mathsf{Elem}_N(X, \langle \Pi\, K\, L\rangle[\langle t\rangle]) \text{ if } K \in \mathsf{Type}_N(X), L \in \mathsf{Type}_N(X.\langle K\rangle), \text{ and } t \in \mathsf{Elem}_N(X, \langle K\rangle).\\
\Pi^n\, k\, l &\in \mathsf{Elem}_N(X, \mathsf{U}_n) \text{ if } k \in \mathsf{Elem}_N(X, \mathsf{U}_n) \text{ and } l \in \mathsf{Elem}_N(X.\langle T_n k\rangle, \mathsf{U}_n).\\
\mathsf{U}^n_m &\in \mathsf{Elem}_N(X, \mathsf{U}_n).\\
t^n_m k &\in \mathsf{Elem}_N(X, \mathsf{U}_n) \text{ if } k \in \mathsf{Neut}(X, \mathsf{U}_m).\\
k &\in \mathsf{Elem}_N(X, \mathsf{U}_n) \text{ and } t \in \mathsf{Elem}_N(X, \langle T_n k\rangle), \text{ if } k \in \mathsf{Neut}(X, \mathsf{U}_n) \text{ and } t \in \mathsf{Neut}(X, \langle T_n k\rangle).
\end{aligned}
$$

Definition 3.3 (Normal forms syntax). If A is in $\mathsf{Type}_M(\langle X\rangle)$, we write $\mathsf{Type}_N(X)|A$ for the subset of normal types K such that $\langle K\rangle = A$.

If a is in $\mathsf{Elem}_M(\langle X\rangle, A)$, we write $\mathsf{Elem}_N(X, A)|a$ for the subset of normal terms such that $\langle k\rangle = u$.

The interpretation $\langle . \rangle$ from Elem_N and Type_N to Elem_M and Type_M can be seen as quotient maps : from syntactical normal terms and types, we associate the corresponding term or type, quotiented up to conversion. Therefore, normalisation amounts to showing that those quotient maps are isomorphisms.

It is important to note that Type_M, $\mathsf{Elem}_M(T)$, Type_N, $\mathsf{Elem}_N(T)$, and $\mathsf{Neut}(T)$ are presheafs, and thus types in the presheaf model $\widehat{\mathcal{C}}$. This will allow us to work internally in the presheaf topos $\widehat{\mathcal{C}}$, and in the following, we will freely use type theoretic notations in this topos.

3.2 Normalisation model

Now, we define the normalisation model, a "proof-relevant" way to express the reducibility candidates: in this model, a type is equipped with a proof of its reducibility, a normal form, a proof that its neutral terms are reducible (i.e a reflect function), and a proof that every reducible term is normalisable (i.e a quote function).

This is described in the following definition.

Definition 3.4 (Reducibility candidates). This definition is internal to the model $\widehat{\mathcal{C}}$.

For T in Type_M, we define $\mathsf{Type}'(T)$ the set of 4-tuples $(T', K, \mathsf{q}_T, \mathsf{r}_T)$ with :

- T' in $\mathsf{Elem}_M(T) \to \mathsf{Set}$
- K is in $\mathsf{Type}_N|T$.
- q_T is in $\Pi_{u:\mathsf{Elem}_M(T)}\, T'\, u \to \mathsf{Elem}_N(T)|u$.
- r_T is in $\Pi_{k:\mathsf{Neut}(T)}\, T'\, \langle k\rangle$.

Wa first define a left-exact morphism from M to $\widehat{\mathcal{C}}$ i.e a morphism of models being strict on the category structure, but weak on context extensions.

Definition 3.5 (Morphism from M to $\widehat{\mathcal{C}}$). To a context Γ in M, we associate a presheaf $|\Gamma|$ in $\widehat{\mathcal{C}}$ by taking $|\Gamma|(X)$ to be the set of substitutions $\langle X\rangle \to \Gamma$ with restriction maps $\rho \mapsto \rho\alpha = \rho\langle\alpha\rangle$ for $\alpha : Y \to_S X$.

Each type A in $\mathsf{Type}_M(\Gamma)$ in the model M defines a presheaf map $|A| : |\Gamma| \to \mathsf{Type}_M$ by $\rho \mapsto A\rho$ (formally, $|A|$ is a presheaf from the category of elements $\int(\Gamma)$, and this is indeed a type in the presheaf model, see [10]).

Each element a in $\mathsf{Elem}_M(\Gamma, A)$ in M defines a global element $|a| : \Pi_{\rho:|\Gamma|}\, \mathsf{Elem}_M(|A|\rho)$.

For any A in $\mathsf{Type}_M(\Gamma)$ in M, we have a constant in the presheaf model :

$$\mathsf{mk} : \Pi_{\rho:|\Gamma|}\, |A|\rho \to |\Gamma.A|$$

with projections $\mathsf{fst} : |\Gamma.A| \to |\Gamma|$ and $\mathsf{snd} : \Pi_{v:|\Gamma.A|}\, |A|(\mathsf{fst}\, v)$ which gives an inverse to the canonical arrow $\langle|\mathsf{p}|, |\mathsf{q}|\rangle : |\Gamma.A| \to |\Gamma|.|A|$. This makes our map $|.| : M \to \widehat{\mathcal{C}}$ a left-exact morphism : we have $|\Gamma.A| \cong |\Gamma|.|A|$ (but not equal!).

In fact, this morphism corresponds directly to the Yoneda embedding : we map the context Γ (object) to its representing presheaf.

With all that, we define operations for normal forms in the presheaf model, with good properties regarding their interpretation. Those will allow us to construct well-behaved type connectives in the normalisation model.

Proposition 3.6. *Let X be a telescope and $\Gamma = \langle X\rangle$ its interpreting context, A in $\mathsf{Type}_M(\Gamma)$ and B in $\mathsf{Type}_M(\Gamma.A)$ two types forming $C = \Pi\, A\, B$.*

Also, let $\rho : |\Gamma|$ be an environment and $K : \mathsf{Type}_N(X)$ such that $\langle K\rangle = |A|\rho$ (a normal form for $|A|\rho$), along with $G : \Pi_{k:\mathsf{Neut}(|A|\rho)}\, \mathsf{Type}_N$ such that $\langle G\, k\rangle = |B|(\mathsf{mk}\, \rho\, \langle k\rangle)$ (normal form for B in the context extended with k).

With all that, we define the following operations :

- *We define $\Pi_S\, K\, G : \mathsf{Type}_N$ such that $\langle \Pi_S\, K\, G\rangle = |C|\rho$.*
- *We define $\lambda_S\, g : \mathsf{Elem}_N(|C|\rho)|w$, for w in $\mathsf{Elem}_M(|C|\rho)$ and $g : \Pi_{k:\mathsf{Neut}(|A|\rho)}\, \mathsf{Elem}_N(|B|(\mathsf{mk}\, \rho\, \langle k\rangle))|(w\, \langle k$*
- *We define $\mathsf{app}_S\, K\, G\, k'\, k : \mathsf{Neut}(|B|(\mathsf{mk}\, \rho\, u))|(w\, u)$ for w in $\mathsf{Elem}_M(|C|\rho)$ and u in $\mathsf{Elem}_M(|A|\rho)$, $k' : \mathsf{Neut}(|C|\rho)|w$ and $k : \mathsf{Elem}_N(|A|\rho)|u$.*

Proof. We detail the construction of the operations for codes.

- For $\lambda_S\, g$, we define : $\lambda_S\, g \;:=\; \lambda(K, G\mathsf{p}_S\, v_0, g\, v_0)$.
  Which satisfies the desired property, because of η-conversion : $\langle\lambda_S g\rangle = \lambda(|A|\rho, |B|(\rho\mathsf{p}, \mathsf{q}), w\, v_0) = w$.
- For app_S, we define : $\mathsf{app}_S\, K\, G\, k'\, k \;:=\; \mathsf{app}(K, G\mathsf{p}_S\, v_0, k', k)$.
  Which satisfies the desired property : $\langle \mathsf{app}_S\, K\, G\, k'\, k\rangle = \mathsf{app}(|A|\rho, |B|(\rho\mathsf{p}, \mathsf{q}), w, u)$.

□

With those operations, we define the normalisation model M^*. This model can be seen as the gluing model along the left-exact morphism $|.|$, with the construction for universes and dependent products described in [11]. In the following definition, we describe the model explicitly, and add the appropriate definitions for Tarski universes.

Definition 3.7 (Normalisation model). A context in M^* is a pair Γ, Γ' where Γ is a context of M and Γ' is a dependent family over $|\Gamma|$ in the presheaf model.

- We define $()'$ to be the constant presheaf $\{0\}$
- We define the extension operation by $(\Gamma, \Gamma').(A, \overline{A}) = \Gamma.A, (\Gamma.A)'$, where $(\Gamma.A)'(\rho, u)$ is defined as the set of pairs ρ', u' with ρ' in $\Gamma'(\rho)$ and u' in $\overline{A}\rho\rho'.1(u)$.

- We define $\mathsf{Type}^*(\Gamma, \Gamma')$ (types of M^*) as the set of pairs $A, \overline{A}$ where A is in $\mathsf{Type}_M(\Gamma)$ and $\overline{A}$ is a global element of :
$$\Pi_{\rho:|\Gamma|}\, \Gamma'(\rho) \to \mathsf{Type}'(|A|\rho)$$
Such that $\overline{A}\rho\rho'$ is a proof of reducibility for A. Hence, every type is equipped with its reducibility proof.
- We define $\mathsf{Elem}^*(\Gamma, (A, \overline{A}))$ (elements of M^*) as the set of pairs $a, \overline{a}$ where a is in $\mathsf{Elem}_M(\Gamma, A)$ and $\overline{a}$ is an element of :
$$\Pi_{\rho:|\Gamma|}\, \Pi_{\rho':\Gamma'(\rho)} \overline{A}\rho\rho'.1(|a|\rho)$$
Such that $\overline{a}\rho\rho'$ is a proof of reducibility for a. Hence, every term is equipped with its reducibility proof.
- We define the dependent product in M^*, denoted $\Pi^*\,(A, \overline{A})\,(B, \overline{B})$, which will be equal to $C, \overline{C}$ with $C = \Pi\, A\, B$ and $\overline{C}\rho\rho' = (R', L, \mathsf{q}_R, \mathsf{r}_R)$ is defined by the following:
  - $R'(w) = \Pi_{u:\mathsf{Elem}_M(|A|\rho)} \Pi_{u':T'(u)} F'uu'(w\, u)$
  - $L = \Pi_S\, K\, G$
  - $\mathsf{q}_R\, w\, w' = \lambda_S\, g$
  - $(\mathsf{r}_R k)uu' = \mathsf{r}_F uu'(\mathsf{app}_S\, K\, G\, k\, (\mathsf{q}_T uu'))$

  with $(T', K, \mathsf{q}_T, \mathsf{r}_T) = \overline{A}\rho\rho'$ and $(F'uu', F_0uu', \mathsf{q}_F uu', \mathsf{r}_F uu') = \overline{B}(\mathsf{mk}\, \rho\, u)(\rho', u')$ for each $u \in Elem_M(|A|\rho)$ and $u' \in T'(u)$, and where G is the function $G\, k = F_0\langle k\rangle(\mathsf{r}_T\, k)$, and g the one which maps $g\, k = \mathsf{q}_F\langle k\rangle(\mathsf{r}_T k)(w\, \langle k\rangle)(w'\langle k\rangle(r_T\, k))$.

Now we can add the constructions related to universes. The existing proof of normalisation by gluing such as the ones in [7, 8] work on a theory with a stratified type sort Type_n and a sort equation $\mathsf{Elem}(\mathsf{U}_n) = \mathsf{Type}_n$. To our knowledge, a rigorous proof of normalisation by gluing of a Tarksi-style theory has not been written down properly.

In fact, the proof is not completely identical. For instance, one needs to embed the normal form of the decoding and lifting of a code inside its computability structure.

Theorem 3.8 (Construction for universes). *The following describes the operations related to universes in the normalisation model, including lifts, codes and decoding.*

- *We define* $\mathsf{U}_n^* = \mathsf{U}_n, \overline{\mathsf{U}_n}$, *with* $\overline{\mathsf{U}_n}\rho\rho' = \mathsf{U}'_n, \mathsf{U}_n, \mathsf{q}_{\mathsf{U}_n}, \mathsf{r}_{\mathsf{U}_n}$ *defined by the following :*
  - U'_n *where* $\mathsf{U}'_n a$ *is the set of* 5*-tuples* $(a', a_0, A_N, a_l, \mathsf{q}_a, \mathsf{r}_a)$ *such that :*
    - $a' : \mathsf{Elem}_M(T_n(a)) \to \mathcal{U}_n$
    - $a_0 : \mathsf{Elem}_N(\mathsf{U}_n)|a$
    - $A_N : \mathsf{Type}_N|T_n(a)$,
    - $a_l : \Pi_{m\geq n}\mathsf{Elem}_N(\mathsf{U}_m)|t_n^m(a)$,
    - $\mathsf{q}_a\, u\, u' : \mathsf{Elem}_N(T_n(a))|u$ *for* $u : \mathsf{Elem}_M(T_n(a))$ *and* $u' : a'u$
    - $\mathsf{r}_a k : a'\, k$ *for* $k : \mathsf{Neut}(T_n(a))$.
  - U_n *is the normal form.*
  - $\mathsf{q}_{\mathsf{U}_n}\, a\, (a', a_0, \mathsf{q}_a, \mathsf{r}_a) = a_0$.
  - $\mathsf{r}_{\mathsf{U}_n} K = (K', K, \mathsf{q}_K, \mathsf{r}_K)$ *where* $K'u$ *is* $\mathsf{Neut}(K)|u$ *and* $\mathsf{q}_K\, u\, u' = u'$, *and* $\mathsf{r}_K\, k = k$.
- *We define* $T_n(a)^* = T_n(a), \overline{T_n(a)}$ *for* a *in* $\mathsf{Elem}_M(X, \mathsf{U}_n)$, *let's write* $\overline{a}\rho\rho' = (a', a_0, A_N, a_l, q_a, r_a)$. *Then we can define :*
  - $R' = a'$
  - $L = A_N$
  - $\mathsf{q}_R = \mathsf{q}_a$
  - $\mathsf{r}_R = \mathsf{r}_a$

- *We define* $t_n^m(a)^* = t_n^m(a), \overline{t_n^m(a)}$ *for a in* $\mathsf{Elem}_M(X, \mathsf{U}_n)$*, let's write* $\overline{a}\rho\rho' = (a', a_0, A_N, a_l, q_a, r_a)$*. Then we can define* $\overline{t_n^m(a)}\rho\rho' = (b', b_0, B_N, q_b, r_b)$ *:*
  - $b' = a'$*, which is well-defined because* $\mathcal{U}_n \subset \mathcal{U}_m$
  - $b_0 = a_l\, m$
  - $B_N = A_N$
  - $q_b = q_a$
  - $r_b = r_a$
- *We define* $(\mathsf{U}_n^m)^* = \mathsf{U}_n^m, \overline{\mathsf{U}_n^m}$ *writing* $\overline{\mathsf{U}_n^m}\rho\rho' = (a', a_0, A_N, q_a, r_a)$ *:*
  - $a'u = Elem_N(\mathsf{U}_n)|u$
  - $a_0 = \mathsf{U}_n^m$
  - $A_N = \mathsf{U}_n$
  - $q_a uu' = u'$
  - $r_a k = (k', k_0, K_N, q_k, r_k)$ *where :*

  $(k'u = Neut(T_n(u)),\ k_0 = k,\ K_N = T_n(k_0),\ k_l\, m' = t_m^{m'}(k_0),\ q_k uu' = u',\ r_k k' = k')$

- *We define* $(\Pi^n\ a\ b)^* = \Pi^n\ a\ b, \overline{\Pi^n\ a\ b}$ *for a in* $\mathsf{Elem}_M(X, \mathsf{U}_n)$ *and b in* $\mathsf{Elem}_M(|T_n a|\rho)$*, let's write* $\overline{a}\rho\rho' = (a', a_0, A_N, q_a, r_a)$ *and* $\overline{b}(\mathsf{mk}\,\rho\, u)(\rho', u') = (b'uu', b_0uu', B_N uu', q_b uu', r_b uu')$ *for every* $u \in \mathsf{Elem}_M(|T_n|\rho)$ *and* $u' \in a'(u)$*. Writing* $\overline{\Pi^n\ a\ b} = (c', c_0, C_N, q_c, r_c)$ *we define:*
  - $c'w = \Pi_{u:\mathsf{Elem}_M(|T_n(a)|\rho)} \Pi_{u':a'(u)} b'uu'(wu)$
  - $c_0 = \Pi^n\ a_0\ b_0$
  - $C_N = \Pi\ A_N\ B_N$
  - $c_l = \Pi\ c_l\ b_l$
  - $q_c ww' = \lambda_S\, g$
  - $(r_c k)uu' = \mathsf{r}_b uu'(\mathsf{app}_S\, a_0\, G\, k\, (q_a uu')$

  Where G is the function $G\, k = B_N\langle k\rangle(\mathsf{r}_a\, k)$*, and g the one which maps* $g\, k = \mathsf{q}_b\langle k\rangle(\mathsf{r}_a k)(w\,\langle k\rangle)(w'\langle k\rangle(r_a\, k$

Proof. The only thing to check is that our construction is well-defined and satisfies the equations required for it to be a model of type theory. □

We now instantiate this construction to the term model M_0, which is the initial model, so that we get an initial map $M_0 \to M_0^*$ which is a section of the projection map $M_0^* \to M_0$. Moreover, as the term model M_0 is the syntactical model, here contexts are the same as telescopes : we have $\langle X\rangle = X$.

Hence, from a context Γ of M_0, we use this section to compute Γ', a context of M_0^*. This Γ' is internally a dependent family over presheafs $|\Gamma|$, but can be seen externally as a family $\Gamma'(\Delta, \rho)$ over Δ and $\rho : \Delta \to \Gamma$, along with restriction maps $\rho' \mapsto \rho'\alpha$ for $\alpha : \Theta \to_S \Delta$.

To conclude, we construct id'_Γ in $\Gamma'(\Gamma, \mathsf{id})$. This is also analogous to Tait's proof, where we also need to instantiate the result using the identity substitution, along with the reducibility candidate for neutral terms, here replaced by r_A.

Lemma 3.9. *By induction :*

- *For* $\Gamma = ()$*, we take* $\mathsf{id}'_\Gamma = 0$ *(as we defined* $()' = 0$*, there is no choice).*
- *If* $\Delta = \Gamma.A$*, we define* $\rho' = \mathsf{id}'_\Gamma \mathsf{p}_S$ *in* $\Gamma'(\Delta, \mathsf{p})$*, and then* $\mathsf{id}'_\Delta = \rho', \mathsf{r}_A(\Delta, \mathsf{p})\rho' v_0$ *which is an element of* $\Delta'(\rho, \mathsf{p})$ *because* $\mathsf{r}_A(\Delta, \mathsf{p})\rho' v_0$ *is an element of* $A'(\Delta, \mathsf{p})\rho'\langle v_0\rangle$*, which is what we wanted.*

In the next section, we will conclude by defining a normalisation by evaluation function.

4 Normalisation by evaluation and consequences

We have the normalisation model instantiated to the term model M_0, along with a projection map $M_0^* \to M$ and its section $M_0 \to M_0^*$. With this, we define a normalisation by evaluation function $\mathsf{nf} : \mathsf{Elem}_M(\Gamma, A) \to \mathsf{Elem}_N(\Gamma, A)$, which is described in the following.

Definition 4.1. For A in $\mathsf{Type}_M(\Gamma)$ and a is in $\mathsf{Elem}_M(\Gamma, A)$ we define :

$$\mathsf{nf}(A) := (\overline{A}\,\mathsf{id}\,\mathsf{id}').2 \qquad\qquad \mathsf{nf}(a) := ((\mathsf{q}_A\,\mathsf{id}\,\mathsf{id}')\,a\,(\overline{a}\,\mathsf{id}\,\mathsf{id}')).1$$

Finally, we get normalisation, by showing that the quotient maps in Elem_N and Type_N are split by nf :

Theorem 4.2 (Normalisation). *For a context Γ, the quotient maps $k \mapsto \langle k \rangle$ and $K \mapsto \langle K \rangle$ (for types and terms) have sections given by* nf.

Proof. We have $\langle \mathsf{nf}(A) \rangle = \langle K \rangle = A$ as K is in $\mathsf{Norm}(A)|K$.

We also have :

$$\langle \mathsf{nf}(a) \rangle = \langle \mathsf{q}_A\,\mathsf{id}\,\mathsf{id}'\,a\,(\overline{a}\,\mathsf{id}\,\mathsf{id}').1 \rangle = a$$

as $q_A\,\mathsf{id}\,\mathsf{id}'$ is in $\Pi_{u:\mathsf{Elem}_M(|A|\rho)} A'\,\mathsf{id}\,\mathsf{id}'\,u \to \mathsf{Elem}_N(A\rho)|u$, and $(\overline{a}\,\mathsf{id}\,\mathsf{id}').1$ is indeed in $A'\,\mathsf{id}\,\mathsf{id}'\,a$. □

It is important to note that this function nf is well defined and can be effectively computed because of the section of the projection $M_0 \to M_0^*$, which comes from the initiality of the term model. We have the following corollary :

Corollary 4.3. *Equality in M_0 is decidable.*

Proof. If a and b are in $\mathsf{Elem}_M(\Gamma, A$, we can compute $\mathsf{nf}(a)$ and $\mathsf{nf}(b)$ in $\mathsf{Elem}_N(\Gamma, A)$, and then check syntactically if $\mathsf{nf}(a) = \mathsf{nf}(b)$ or not.
Then, as $\mathsf{nf}(a) = \mathsf{nf}(b)$ if and only if $a = b$, we can decide if $a = b$ or not. □

4.1 Injectivity of type formers

In this section we show the injectivity of dependent product (in the term model), using a semantic argument in the normalisation model. First we need the following essential lemma.

Lemma 4.4. nf *commutes with the dependent product operation.*

Proof. We use the definitions: $\mathsf{nf}(\Pi(A, B)) = \Pi_S\,K\,G = \Pi\,K\,(G\,\mathsf{p}_S\,v_0) = \Pi\,K\,(F_0\,\langle v_0 \rangle\,(\mathsf{r}_T\,v_0))$.

Where $\overline{A}\,\mathsf{id}\,\mathsf{id}' = (T', K, \mathsf{q}_T, \mathsf{r}_T)$ and $(F' v_0 v_0', F_0 v_0 v_0', \mathsf{q}_F v_0 v_0', \mathsf{r}_F v_0 v_0') = \overline{B}(\mathsf{mk}\,\mathsf{id}\,v_0)(\mathsf{id}', v_0')$.

So our calculation gives the desired : $\mathsf{nf}(\Pi(A, B)) = \Pi\,\mathsf{nf}(A)\,\mathsf{nf}(B)$. □

From this, we deduce Π-injectivity really easily :

Theorem 4.5 (Injectivity of dependent product). *The dependent product (in M_0) is injective.*

Proof. Let's assume that $\Pi\,A\,B = \Pi\,A'\,B'$ in M_0. Then, we have

$$\mathsf{nf}(\Pi\,A\,B) = \mathsf{nf}(\Pi\,A'\,B')$$

i.e

$$\Pi(\mathsf{nf}\,A)(\mathsf{nf}\,B) = \Pi(\mathsf{nf}\,A')(\mathsf{nf}\,B')$$

Thus, as Π is injective in Type_N (it is a syntactic operation), we get : $\mathsf{nf}(A) = \mathsf{nf}(A')$ and $\mathsf{nf}(B) = \mathsf{nf}(B')$, and so $A = [\mathsf{nf}(A)] = [\mathsf{nf}(A')] = A'$. Likewise, we also have $B = B'$. □

To prove our equivalence results in the next section, we will also need a injectivity for universes, which is stated by the following theorem.

Theorem 4.6 (Injectivity of universes). *If we have* $\mathsf{U}_{n_0} = \mathsf{U}_{n_1}$ *in* M_0*, then* $n_0 = n_1$.

Proof. We have $\mathsf{U}_{n_0} = \mathsf{U}_{n_1}$ in M_0, so $\mathsf{nf}(\mathsf{U}_{n_0}) = \mathsf{nf}(\mathsf{U}_{n_1})$ in the normalisation model M^*, which corresponds to $\mathsf{U}_{n_0} = \mathsf{U}_{n_1}$ in M^*, and as this equality is syntactic we get $n_0 = n_1$. □

Subject reduction for β-conversion also follows from Π-injectivity.

4.2 Subject reduction

We are going to show an essential property of type theory : subject reduction. It corresponds to a form of "type correctness", and to the static specification of a program that has to be stable by execution (which corresponds here to reduction).

Theorem 4.7. *If* $\Gamma \vdash a = b : A$ *and* $\Gamma \vdash a : A$ *then we have a derivation* $\Gamma \vdash b : A$.

Proof. We show the property by induction on the term a. We will only consider the interesting case where a is a β-redex i.e $a = (\lambda(x : A_1).t)u : A$ and $b = t[u]$.

Then by inverting the derivation, we necessarily have types B_1, B_2, A_2 and derivations $\Gamma, x : A_1 \vdash t : B_1$, $\Gamma \vdash u : A_2$, $\Gamma \vdash B_2[u] = A$ and $\Gamma \vdash \Pi(x : A_1)B_1 = \Pi(x : A_2)B_2$.

By Π-injectivity we thus have $\Gamma \vdash A_2 = A_1$ and $\Gamma, x : A_1 \vdash B_1 = B_2$. Finally, by combining the derivations, we obtain $\Gamma, x : A_1 \vdash t : B_2$ and $\Gamma \vdash u : A_1$, and hence : $\Gamma \vdash t[u] : B_2[u]$.

We then conclude with $B_2[u] = A$ that gives us the desired $\Gamma \vdash t[u] : A$ □

With all of those result, we will be able to show that the equivalence of all our systems.

5 Equivalence results

In this section, we show that all the systems $\mathcal{T}_G$, $\mathcal{T}_T$, $\mathcal{T}_R$ and $\mathcal{T}_P$ are equivalent. In particular, we show that Tarski-style and Russell-style universes are equivalent.

5.1 Equivalence of $\mathcal{T}_G$ and $\mathcal{T}_T$: explicit and implicit substitutions

We show by an elementary proof that $\mathcal{T}_T$ and $\mathcal{T}_G$ are equivalent, and thus, $\mathcal{T}_T$ can be rigorously interpreted as a CWF.

First, we would have to show that we can remove context annotations from the substitutions in $\mathcal{T}_G$ formally. This can be done with an analogous technique as for the equivalence of Russell and Tarski universes, and the main point is that the contexts and types can be inferred from the derivation. We will not detail this proof here.

Then, we define a from $\mathcal{T}_G$ to $\mathcal{T}_T$ that creates a judgment in $\mathcal{T}_T$ with implicit substitutions. For this, we map substitutions in $\mathcal{T}_G$ to a list of "elementary" substitutions in $\mathcal{T}_T$, as there is no composition in this system. Then, when translating the application of a substitution to a term or a type, we can fold that list to get a type formed by applying all those substitutions successively.

Definition 5.1. We define $\mathsf{proj}(.)$ by induction:

$$\begin{aligned}
\mathsf{proj}(\Gamma \vdash J) &= \mathsf{proj}(\Gamma) \vdash \mathsf{proj}(J) & \mathsf{proj}(\mathsf{q}) &= v_0 \\
\mathsf{proj}(\sigma : \Gamma \to \Delta) &= \vdash & \mathsf{proj}(\mathsf{p}) &= \mathsf{p} \\
\mathsf{proj}([a]) &= [\mathsf{proj}(a)] & \mathsf{proj}(\mathsf{id}) &= \\
\mathsf{proj}(A\sigma) &= (...(\mathsf{proj}(A)\sigma_1)...)...\sigma_n & \mathsf{proj}(\sigma\delta) &= \mathsf{proj}(\sigma) :: \mathsf{proj}(\delta) \\
\mathsf{proj}(a\sigma) &= (...(\mathsf{proj}(a)\sigma_1)...)...\sigma_n & \mathsf{proj}(\sigma^+) &= (\sigma_1^+, ..., \sigma_n^+)
\end{aligned}$$

Proof. We check that this definition gives types and term in $\mathcal{T}_T$, which is not completely clear with what is going on in the cases $A\sigma$ and $a\sigma$.

In fact, we check that $\mathrm{proj}(\sigma)$ gives a list of substitutions which are valid syntactically in $\mathcal{T}_T$. This can be done easily by induction, but we ommit the details here. □

For the equivalence result, we need the usual subsitution lemma in $\mathcal{T}_T$:

Lemma 5.2 (Substitution lemma). *If $\Gamma \vdash J$ in $\mathcal{T}_T$ and $\alpha : \Delta \to \Gamma$ is a substitution, then we have a derivation : $\Gamma \vdash J\alpha$.*

Proof. By direct induction. □

With all of this, we show the following admissibility result :

Lemma 5.3. *The erasure of rules in $\mathcal{T}_G$ are admissible in $\mathcal{T}_T$.*

Proof. Direct with the substitution lemma, which we iterate when we unfold the list of "elementary" substitutions created by erase. For example for the rule :

$$\frac{\Gamma \vdash A\,\mathsf{type} \qquad \vdash \sigma : \Delta \to \Gamma}{\Delta \vdash A\sigma\,\mathsf{type}}$$

the erasure is just :

$$\frac{\Gamma \vdash \mathsf{proj}(A)\,\mathsf{type}}{\Delta \vdash \mathsf{proj}(A)\sigma_1 ... \sigma_n\,\mathsf{type}}$$

Where $\mathrm{proj}(\sigma) = (\sigma_1, ..., \sigma_n)$, and this derivation is admissible with n applications of the substitution lemma. □

Consequently, we get the following correction result :

Theorem 5.4. *If $\Gamma \vdash J$ in $\mathcal{T}_G$ then $\mathrm{proj}(\Gamma \vdash J)$ is derivable in $\mathcal{T}_T$.*

Proof. By direct induction with the lemma, we replace the rules by the admissible ones. □

Finally, we deduce that the two systems are equivalent :

Theorem 5.5. *The two systems $\mathcal{T}_G$ and $\mathcal{T}_T$ are equivalent.*

Proof. Each rule of $\mathcal{T}_T$ is trivially admissible of $\mathcal{T}_G$ because $\mathcal{T}_T$ is a subsystem of $\mathcal{T}_G$.

Hence, we have the identity map from $\mathcal{T}_T$ and $\mathcal{T}_G$, and as proj acts identically on types and terms without substitution, we get our equivalence result. □

Hence, we can use implicit substitutions freely with the system "à la Tarski", while still being able to give a rigorous interpretation in a CWF model. This proves the equivalence of $\mathcal{T}_T$ and the GAT presentation of type theory (here $\mathcal{T}_G$). An alternative approach could use the initiality result of [9] for contextual categories, which may be adapted to interpret $\mathcal{T}_T$ as a CWF.

5.2 Equivalence of $\mathcal{T}_T$ and $\mathcal{T}_R$: Tarski and Russell universes

With the essential properties of $\mathcal{T}_T$ shown in section 4, we prove the equivalence of Tarski-style and Russell-style universes. First, we need to consider both theories with type annotations the derivation rules are those corresponding to the equational theory in section 2.

For that we consider the following erasure operation, which forms a forgetful map from Tarski universes to Russell ones.

Definition 5.6 (Erasure). We define the erasure operation, denoted $|.|$:

$$
\begin{aligned}
|\mathsf{U}_n| &= U_n \\
|\mathsf{U}^n_m| &= \mathsf{U}_m \\
|\mathsf{app}(A, B, c, a)| &= \mathsf{app}(|A|, |B|, |c|, |a|) \\
|\lambda(A, B, b)| &= \lambda(|A|, |B|, |b|) \\
|v_i| &= v_i
\end{aligned}
\qquad
\begin{aligned}
|T_n(a)| &= |a| \\
|t^n_m(a)| &= |a| \\
|\Pi^n\ a\ b| &= \Pi(|a|, |b|) \\
|\Pi(A, B)| &= \Pi(|A|, |B|)
\end{aligned}
$$

We now show that this operation indeed preserves derivations.

Lemma 5.7 (Correction of erasure). *If $\Gamma \vdash J$ in $\mathcal{T}_T$, then $|\Gamma| \vdash |J|$ in $\mathcal{T}_R$.*

Proof. By direct induction. □

Then, we want to show that this forgetful map has an inverse, hence proving the equivalence of the two systems. For this, we first show a useful theorem.

For this we will need some technical lemmas.

Lemma 5.8. *If we have a derivation $\Gamma, A \vdash J$, where J is an arbitrary judgment, then we necessarily have a derivation $\Gamma \vdash A$ type.*

Proof. We prove the result by induction on the derivation $\Gamma, A \vdash J$. □

And similarly, we have the other lemma :

Lemma 5.9. *If we have a derivation $\Gamma \vdash a : A$ then we necessarily have a derivation $\Gamma \vdash A$ type.*

Proof. By induction on $\Gamma \vdash a : A$. It is important to note that the obtained derivation is not necessarily a sub-derivation of $\Gamma \vdash a : A$ (if A is a universe for example). □

Those two lemmas allow us to consider rules in $\mathcal{T}_T$ with a premise $\Gamma \vdash A$ type for all rules that require it. Indeed, it will be essential for us to apply the induction hypothesis in the proof of equivalence. Everything is condensed in the following lemma :

Lemma 5.10. *We can remove the typing premises to the application, abstraction and Π^n rule. This new system is equivalent to the previous one.*

In the following proof of proposition C.5, we will use freely that we have the typing hypothesises in the rule, but we will use the new rules to simplify the notations.

We also need the following lemma to modify derivations when we have a conversion judgment.

Lemma 5.11. *If we have a derivation $\Gamma, A \vdash J$ and a convertibility derivation $\Gamma \vdash A = B$, then we construct a derivation $\Gamma, B \vdash J$.*

Proof. It results from the fact that $=$ is a congruence, and of the rule that gives $a : B$ when $a : A$ and $A = B$. Then, it is shown by direct induction. □

We also need this lemma that will be useful in the proof of the main lemma.

Lemma 5.12 ( ). *Let's assume that : $\Gamma \vdash A_0 = \mathsf{U}_{n_0}$, $\Gamma \vdash A_1 = \mathsf{U}_{n_1}$ and $\Gamma \vdash u_0 = t^{n_0}_k(v_0) : A_0$, $\Gamma \vdash u_1 = t^{n_1}_k(v_1) : A_1$ with $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$.*

Then if $\Gamma \vdash A_0 = A_1$, we have : $\Gamma \vdash u_0 = u_1 : A_0$.

Proof. Mostly direct using the conversion rules for liftings in $\mathcal{T}_T$. See the detailed proof in appendix B.1. □

Now we state the main result, which is adapted from chapter 4 of [18], where he proves metatheoretical results.

Proposition 5.13 ( ). *If* $\Gamma \vdash A$ type, $\Gamma \vdash B$ type *in* $\mathcal{T}_T$ *and* $|A| = |B|$, *then* $\Gamma \vdash A = B$.
If $\Gamma \vdash u_0 : A_0$ *and* $\Gamma \vdash u_1 : A_1$ *in* $\mathcal{T}_T$ *and* $|u_0| = |u_1|$ *then*

- *Either* $\Gamma \vdash A_0 = A_1$ *and* $\Gamma \vdash u_0 = u_1 : A_0$.
- *Or* $\Gamma \vdash A_0 = \mathsf{U}_{n_0}$, $\Gamma \vdash A_1 = \mathsf{U}_{n_1}$ *and* $\Gamma \vdash u_0 = t_k^{n_0}(v_0) : A_0$, $\Gamma \vdash u_1 = t_k^{n_1}(v_1) : A_1$ *with* $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$.

Proof. We prove the property by mutual induction, proving both statements at the same time. One of the key property used is the injectivity of type formers. For instance, when treating the app case, we get an equality of Π-types by induction hypothesis, and need to deduce the convertibility of the two composing types. The proof also makes an extensive use of lemma 5.11 to construct derivation with the appropriate contexts when we have two premises. See appendix B.1 for the detailed proof. □

Hence, we deduce the following corollary, which states that if we have two judgments in $\mathcal{T}_T$ which are syntactically equal in $\mathcal{T}_R$ (when removing coercions, liftings...), then they are provably convertible in $\mathcal{T}_T$. Hence, the Tarski system is not much stronger than the Russell one.

Corollary 5.14 (Judgemental injectivity of erasure). *If* $\Gamma \vdash u_0 : A_0$ *and* $\Gamma \vdash u_1 : A_1$ *in* $\mathcal{T}_T$ *and* $|u_0| = |u_1|$, *with the additional hypothesis* $|A_0| = |A_1|$, *then :* $\Gamma \vdash A_0 = A_1$ *and* $\Gamma \vdash u_0 = u_1 : A_0$.

Proof. This is a direct application of the two previous lemmas. □

Now we prove the following theorem, stating the formal equivalence between the two systems:

Theorem 5.15 (Section of erasure). *If* $\Gamma \vdash J$ *in the system* $\mathcal{T}_R$, *then there exists a unique, up to conversion judgment* $\Gamma_1 \vdash J_1$ *in the system* $\mathcal{T}_T$ *such that* $|\Gamma_1| = \Gamma$ *and* $|J_1| = J$.

Proof. Uniqueness up to conversion follows directly from the previous lemma. We prove existence by induction, by making an extensive use of the previous lemma for rules with two premises. Indeed, when using the induction hypothesis on two premises, we need to make sure that the same types have the same translations. See detailed proof in appendix. □

Hence, we showed the equivalence of the two systems, and thus we can give categorial semantics rigorously to the system $\mathcal{T}_R$ "à la Russell", with CWFs.

5.3 Equivalence of $\mathcal{T}_R$ and $\mathcal{T}_P$: finally GAT and PTS

We define the strip function, which erases type annotations from a term of the system $\mathcal{T}_R$, and gives a term of $\mathcal{T}_P$. This function is analogous to the one defined in [18].

Definition 5.16. strip is defined inductively by :

- $\mathsf{strip}(\lambda(A, B, b)) = \lambda\ (A, \mathsf{strip}(b))$
- $\mathsf{strip}(\mathsf{app}(A, B, c, a)) = \mathsf{strip}(c)\ \mathsf{strip}(a)$.
- $\mathsf{strip}(\Pi(A, B)) = \Pi(\mathsf{strip}(A), \mathsf{strip}(B))$.
- $\mathsf{strip}(\mathsf{U}_n) = \mathsf{U}_n$

We have quite easily :

Theorem 5.17. *If* $\Gamma \vdash J$ *in* $\mathcal{T}_R$, *then we have a derivation* $\mathsf{strip}(\Gamma) \vdash \mathsf{strip}(J)$ *in* $\mathcal{T}_P$.

Proof. We just need to prove that the stripping of rules in $\mathcal{T}_R$ are admissible in $\mathcal{T}_P$, and we just need to inspect each rule. Then, the theorem is deduced by an induction on the derivation which replaces rules in $\mathcal{T}_R$ by the admissible ones in $\mathcal{T}_P$. □

Hence, like in [18], we prove the following lemma.

Lemma 5.18. *We have :*

- *If* $\Gamma \vdash u_0 : A_0$ *and* $\Gamma \vdash u_1 : A_1$ *in* $\mathcal{T}_R$*, with* $\mathsf{strip}(u_0) = \mathsf{strip}(u_1)$ *in* $\mathcal{T}_P$*, then* $\Gamma \vdash u_0 = u_1 : A_0$*.*
- *If* $\Gamma \vdash A\,\mathsf{type}$ *and* $\Gamma \vdash B\,\mathsf{type}$ *with* $\mathsf{strip}(A) = \mathsf{strip}(B)$*, then we have* $\Gamma \vdash A = B$*.*

Proof. Here, again we prove the two statements simultaneously, by mutual induction. The proof is similar to the one of proposition C.5. See appendix for the detailed proof. □

And we show the following theorem :

Theorem 5.19. *If* $\Gamma \vdash J$ *in* $\mathcal{T}_P$*, then we have a unique up to conversion judgment* $\Gamma' \vdash J'$ *in* $\mathcal{T}_R$ *such that* $\mathsf{strip}(\Gamma') = \Gamma$ *and* $\mathsf{strip}(J') = J$*.*

Proof. By induction on the derivation $\Gamma \vdash J$, with the previous lemma, we construct a derivation in $\mathcal{T}_R$. The main point is that the typing annotations can be inferred from the derivation, and that the previous lemma gives us uniqueness for rules with two premises. See appendix for the detailed proof. □

We can also relax the rules of our system $\mathcal{T}_P$ to remove "A type" judgments. In fact, we observe that the following rules are redundant.

Lemma 5.20. *The rules for* $\Gamma \vdash \mathsf{U}_n\,\mathsf{type}$ *and* $\Gamma \vdash \Pi(A, B)\,\mathsf{type}$ *are redundant because of the following rule :*

$$\frac{\Gamma \vdash A : \mathsf{U}_n}{\Gamma \vdash A\,\mathsf{type}}$$

Proof. For the rule :

$$\frac{\dfrac{\Pi_1}{\Gamma \vdash C\,\mathsf{type}} \qquad \dfrac{\Pi_2}{\Gamma, C \vdash D\,\mathsf{type}}}{\Gamma \vdash \Pi(C, D)\ \mathsf{type}}$$

When we translate the premises, we get $\Gamma \vdash C : \mathsf{U}_{n_1}$ and $\Gamma, C \vdash D : \mathsf{U}_{n_2}$, with not necessarily $n_1 = n_2$. By taking $n := \max(n_1, n_2)$, and replacing the premises with the appropriate rules for cumulativity, we get $\Gamma \vdash \Pi\, C\, D : \mathsf{U}_n$. □

Hence, we remove those rules in the system $\mathcal{T}_R$, and get a system with no "well-formed type" judgments ("A type"). We can also consider named variables instead of de Bruijn indices. Hence, we get a theory really close to a pure type system, with the additional rule for cumulativity :

$$\frac{\Gamma \vdash A : \mathsf{U}_n}{\Gamma \vdash A : \mathsf{U}_p}\ (n < p)$$

6 Extending the results

One thing to mention, is that in the system with Russell universes, typing is not closed under η reduction, as shown in the following :

Example 6.1. In the context $x : \mathsf{U}_1 \to \mathsf{U}_1$, we have $\lambda_{y:\mathsf{U}_0}(xy) : \mathsf{U}_0 \to \mathsf{U}_2$ but we *do not have* $x : \mathsf{U}_0 \to \mathsf{U}_2$.

However, we believe that this property is not that essential for a type theory.

We can extend the proof of normalisation to a theory with identity types. Hence, we still get injectivity of Π-types and universes, which were the crucial lemmas for our equivalence result.

Hence, the proof directly adapts to a theory with univalence taken as an axiom i.e where we work in the context $U \vdash$.

It would also be great if the equivalence result still holds when adding general inductive types. However, a thing that could go wrong would be canonicity, according to [14]. But the counter-example he provides makes use of the fact that subtyping propagates along constructors : $A \leq B$ implies $\mathsf{list}(A) \leq \mathsf{list}(B)$, so for our system with cumulative universes it would mean that $\mathsf{list}(\mathsf{U}_0) \leq \mathsf{list}(\mathsf{U}_1)$. This breaks canonicity, and allow to define a coercion function $c^l_k : \mathsf{list}(\mathsf{U}_k) \to \mathsf{list}(\mathsf{U}_l)$ which do not satisfy $c^2_1(c^1_0(l)) = c^2_0(l)$.

Hence, we believe that such a subtyping propagation is not desirable for our theory. This is also true for Π-types, the relation $B \leq B'$ implying $\Pi(A, B) \leq \Pi(A, B')$ is not true in the Set model, or at least unclear depending on the encoding of application in the set theory. If we do not impose such propagation of cumulativity, the equivalence proof still holds and we still have canonicity, as the proof of [7] extends to general inductive types. However, there are some cases where structural subtyping could be useful, as described in the paper [12].

When adding an impredicative universe of proof-irrelevant propositions such as it is done in [8], we still get the equivalence result for such a system, which is close to the one used in Lean.

6.1 Universe polymorphism

We want to extend the equivalence result to a theory with universe polymorphism. In fact, one often wants to prove theorems uniformly for arbitrary universes. Thus, we need a way to describe universe levels and constraints explicitly, while keeping decidability of equality. The system described in [3] fulfills those requirements. Nevertheless, normalisation for such a system has not been proven yet, even if it is conjectured that it could be proven by an extension of the gluing proof, with a notion of model that will be described in a forthcoming paper from Bezem et al.

If we assume that such a proof holds, we get injectivity of type formers, and we adapt our equivalence proof to a system with universe polymorphism. In fact, we just have to look at the rules closely to make sure that the inductions will still work. This work is done in appendix C.

7 Conclusion

To conclude, we proved that the GAT and PTS presentations of type theory are equivalent, and that the latter is also initial as CWF models. Notably, Tarski and Russell presentations of cumulative universes are equivalent for a theory with $\beta\eta$ judgmental conversion and common inductive types, and this work extend with universe polymorphism.

Those equivalence result are fundamental to justify the informal use of Russell universes in type theory. Also, this new proof relies on normalisation, which is a strong result, yet it is still elementary and modular. We believe that it could be adapted to many extensions of the theory presented in this paper.

References

[1] Arthur Adjedj, Meven Lennon-Bertrand, Thibaut Benjamin, and Kenji Maillard. Adaptt: Functoriality for dependent type casts. *Proc. ACM Program. Lang.*, (POPL), 2026.

[2] Ali Assaf. A calculus of constructions with explicit subtyping. *TYPES*, 2014.

[3] Marc Bezem, Thierry Coquand, Peter Dybjer, and Martín Escardó. Type theory with explicit universe polymorphism. *TYPES*, 2023.

[4] Marc Bezem, Thierry Coquand, Peter Dybjer, and Martín Escardó. A Generalized Algebraic Theory for Type Theory with Explicit Universe Polymorphism. 2026.

[5] John Cartmell. Generalised algebraic theories and contextual categories. *Annals of Pure and Applied Logic*, pages 209–243, 1986.

[6] Simon Castellan, Pierre Clairambault, and Peter Dybjer. Categories with families: Unityped, simply typed, and dependently typed. *CoRR*, 2019.
[7] Thierry Coquand. Canonicity and normalisation for dependent type theory. *CoRR*, 2019.
[8] Thierry Coquand. Reduction free normalisation for a proof irrelevant type of propositions. *CoRR*, 2021.
[9] Menno de Boer. A proof and formalization of the initiality conjecture of dependent type theory, 2020.
[10] Martin Hofmann. *Syntax and semantics of dependent types*, pages 13–54. 1997.
[11] Ambrus Kaposi, Simon Huber, and Christian Sattler. Gluing for Type Theory. *FSCD*, 2019.
[12] Théo Laurent, Meven Lennon-Bertrand, and Kenji Maillard. Definitional functoriality for dependent (sub)types – extended version. *CoRR*, 2024.
[13] Meven Lennon-Bertrand. What Does It Take to Certify a Conversion Checker? In *FSCD*, 2025.
[14] Zhaohui Luo. Notes on universes in type theory, 2012.
[15] Per Martin-Löf. An intuitionistic theory of types: Predicative part. 1975.
[16] Per Martin-Löf. Constructive mathematics and computer programming. 1982.
[17] Per Martin-Löf. Intuitionistic type theory. *Bibliopolis*, 1984.
[18] Thomas Streicher. *Semantics of Type Theory: Correctness, Completeness and Independence Results.* 2012.
[19] François Thiré. *Interoperability between proof systems using the logical framework Dedukti.* PhD thesis, 2020.

A Appendix : Type systems

A.1 The system $\mathcal{T}_T$: Tarski universes

We specify the system $\mathcal{T}_T$ with derivation rules, which directly translates the equations of section 2. This formulation with rules will be useful to prove properties by induction. We start with the "typing" judgments $a : A$ and the well-formed type judgment A type :

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma.A \vdash v_0 : A\mathsf{p}} \qquad \frac{\Gamma \vdash v_n : A \qquad \Gamma \vdash B \text{ type}}{\Gamma.B \vdash v_{n+1} : A\mathsf{p}} \qquad \frac{\Gamma \vdash A \text{ type} \qquad \Gamma.A \vdash B \text{ type}}{\Gamma \vdash \Pi(A,B) \text{ type}}$$

$$\frac{\Gamma.A \vdash b : B \qquad \Gamma.A \vdash B \text{ type} \qquad \Gamma \vdash A \text{ type}}{\Gamma \vdash \lambda(A,B,b) : \Pi(A,B)} \qquad \frac{\Gamma \vdash c : \Pi(A,B) \qquad \Gamma.A \vdash a : A \qquad \Gamma.A \vdash B \text{ type} \qquad \Gamma \vdash}{\Gamma \vdash \mathsf{app}(A,B,c,a) : B[a]}$$

$$\frac{}{\Gamma \vdash \mathsf{U}_n \text{ type}} \qquad \frac{\Gamma \vdash a : \mathsf{U}_n \qquad \Gamma.T_n(a) \vdash b : \mathsf{U}_n \qquad \Gamma \vdash T_n(a) \text{ type}}{\Gamma \vdash \Pi^n\, a\, b : \mathsf{U}_n}$$

$$\frac{\Gamma \vdash a : \mathsf{U}_n}{\Gamma \vdash T_n(a) \text{ type}} \qquad \frac{}{\Gamma \vdash \mathsf{U}^n_m : \mathsf{U}_n}\ (m < n) \qquad \frac{\Gamma \vdash a : \mathsf{U}_n}{\Gamma \vdash t^n_m(a) : \mathsf{U}_p}\ (n < p) \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash A = B}{\Gamma \vdash a : B}$$

We also have the following conversion rules :

$$\frac{\Gamma \vdash a = a' : A \qquad \Gamma \vdash A = B}{\Gamma \vdash a = a' : B} \qquad \frac{\Gamma \vdash A = B \qquad \Gamma \vdash A = C}{\Gamma \vdash B = C} \qquad \frac{A \text{ type}}{A = A}$$

$$\frac{\Gamma \vdash a = b : A \qquad \Gamma \vdash a = c : A}{\Gamma \vdash b = c : A} \qquad \frac{a : A}{a = a : A} \qquad \frac{\Gamma \vdash A = A' \qquad \Gamma.A \vdash B = B'}{\Gamma \vdash \Pi(A,B) = \Pi(A',B')}$$

$$\frac{\Gamma.A \vdash b : B \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathsf{app}(A,B,\lambda(A,B,b),a) = b[a] : B[a]}\ (\beta\text{-rule}) \qquad \frac{f : \Pi\, A\, B}{f = \lambda(A,B,\mathsf{app}(A\mathsf{p}, B\mathsf{p}^+, f\mathsf{p}, v_0)) : \Pi(A,B)}\ (\eta\text{-rule})$$

$$\frac{\Gamma \vdash a = a' : \mathsf{U}_n}{\Gamma \vdash T_n(a) = T_n(a')} \qquad \frac{\Gamma \vdash a = a' : \mathsf{U}_n}{\Gamma \vdash t^p_n(a) = t^p_n(a') : \mathsf{U}_p}\ (n < p)$$

$$\frac{\Gamma \vdash a : \mathsf{U}_n \qquad \Gamma, T_n(a) \vdash b : \mathsf{U}_n}{T_n(\Pi^n\, a\, b) = \Pi(T_n(a), T_n(b))} \qquad \frac{\Gamma \vdash a : \mathsf{U}_n \qquad \Gamma, T_n(a) \vdash b : \mathsf{U}_n}{t_n^p(\Pi^n\, a\, b) = \Pi^p(t_n^p(a), t_n^p(b))}\,(n < p)$$

$$\frac{}{\Gamma \vdash T_n(\mathsf{U}_m^n) = \mathsf{U}_m} \qquad \frac{}{\Gamma \vdash t_n^p(\mathsf{U}_m^n) = \mathsf{U}_m^p : \mathsf{U}_p}\,(n < p)$$

$$\frac{\Gamma \vdash a : \mathsf{U}_m}{\Gamma \vdash T_n(t_m^n(a)) = T_m(a)}\,(n < m) \qquad \frac{\Gamma \vdash a : \mathsf{U}_m}{\Gamma \vdash t_n^p(t_m^n(a)) = t_m^p(a) : \mathsf{U}_p}\,(m < n < p)$$

We also require the rules that makes = a congruence, even if we don't precise them all.

A.2 The system $\mathcal{T}_G$: explicit substitutions

The system $\mathcal{T}_G$ is really close to $\mathcal{T}_T$, the only difference is that substitiutions are explicit : they appear directly in the syntax. Hence, the syntax is given by :

$$\Gamma, \Delta ::= () \mid \Gamma.A$$

$$A, B ::= \Pi\ A\ B \mid \mathsf{U}_n \mid T_n(a) \mid A\sigma$$

$$a, b ::= \mathsf{app}(A, B, c, a) \mid \lambda(A, B, b) \mid t_m^n(a) \mid \mathsf{U}_m^n \mid \Pi^n\, a\, b \mid \mathsf{q}_\Gamma \mid a\sigma$$

$$\sigma, \delta ::= \sigma\delta \mid \mathsf{id}_\Gamma \mid \mathsf{p}_{\Gamma,\Gamma.A} \mid [a] \mid \sigma^{+(A)}$$

The rules are the same as $\mathcal{T}_T$, with only one modification for variables, that are replaced by the rule for q :

$$\frac{\Gamma \vdash A\ \mathsf{type}}{\Gamma.A \vdash q_{\Gamma.A} : A\mathsf{p}_{\Gamma,\Gamma.A}}$$

Then, we have introduction rules for substitutions :

$$\frac{\Gamma \vdash}{\vdash \mathsf{id}_\Gamma : \Gamma \to \Gamma} \qquad \frac{\Gamma \vdash A\ \mathsf{type}}{\vdash \mathsf{p}_{\Gamma,\Gamma.A} : \Gamma.A \to \Gamma} \qquad \frac{\Gamma \vdash a : A}{\vdash [a] : \Gamma \to \Gamma.A}$$

$$\frac{\vdash \sigma : \Delta \to \Gamma \qquad \Gamma \vdash A\ \mathsf{type}}{\vdash \sigma^{+(A)} : \Delta.A\sigma \to \Gamma.A} \qquad \frac{\vdash \sigma : \Delta \to \Gamma \qquad \vdash \delta : \Theta \to \Delta}{\vdash \sigma\delta : \Theta \to \Gamma} \qquad \frac{\vdash \sigma : \Delta \to \Gamma \qquad \Gamma \vdash A\ \mathsf{type}}{\Delta \vdash A\sigma\ \mathsf{type}} \qquad \frac{\vdash \sigma : \Delta \to \Gamma \qquad \Gamma \vdash t : A}{\Delta \vdash t\sigma\ \mathsf{type}}$$

And the conversion rules for substitutions :

$$\frac{\vdash \sigma : \Delta \to \Gamma}{\vdash \sigma\mathsf{id}_\Delta = \sigma : \Delta \to \Gamma} \qquad \frac{\vdash \sigma : \Delta \to \Gamma}{\vdash \mathsf{id}_\Gamma\sigma = \sigma : \Delta \to \Gamma} \qquad \frac{\Gamma \vdash a = a' : A}{\vdash [a] = [a'] : \Gamma \to \Gamma.A}$$

$$\frac{\vdash \sigma : \Delta \to \Gamma \qquad \vdash \delta : \Theta \to \Delta \qquad \vdash \nu : \Xi \to \Theta}{\vdash (\sigma\delta)\nu = \sigma(\delta\nu) : \Xi \to \Gamma} \qquad \frac{\vdash \sigma = \delta : \Delta \to \Gamma \qquad \vdash \sigma = \nu : \Delta \to \Gamma}{\vdash \sigma = \nu : \Delta \to \Gamma}$$

$$\frac{\vdash \sigma = \sigma' : \Delta \to \Gamma \qquad \vdash \delta : \Theta \to \Delta}{\vdash \sigma\delta = \sigma'\delta : \Theta \to \Gamma} \qquad \frac{\vdash \sigma = \sigma' : \Delta \to \Gamma \qquad \vdash \delta : \Gamma \to \Theta}{\vdash \delta\sigma = \delta\sigma' : \Delta \to \Theta} \qquad \frac{\Gamma \vdash \Pi(A, B)\ \mathsf{type} \qquad \vdash \sigma : \Delta \to \Gamma}{\Delta \vdash \Pi(A, B)\sigma = \Pi(A\sigma, B\sigma^{+(A)})}$$

$$\frac{\Gamma \vdash \lambda(A,B,b) : \Pi(A,B) \qquad \vdash \sigma : \Delta \to \Gamma \qquad \Gamma \vdash \mathsf{app}(A,B,c,a) : B[a] \qquad \vdash \sigma : \Delta \to \Gamma}{\Delta \vdash \lambda(A,B,b)\sigma = \lambda(A\sigma, B\sigma^{+(A)}, b\sigma^{+(A)}) : \Pi(A\sigma, B\sigma^{+(A)}) \qquad \Delta \vdash \mathsf{app}(A,B,c,a)\sigma = \mathsf{app}(A\sigma, B\sigma^{+(A)}, c\sigma, a\sigma) : B\sigma[a\sigma]}$$

A.3 The system $\mathcal{T}_R$: Russell with annotations

We specify the system $\mathcal{T}_R$, which is similar to $\mathcal{T}_T$, the only difference being that this system has Russell universes instead of Tarski ones.

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma.A \vdash v_0 : A\mathsf{p}} \qquad \frac{\Gamma \vdash v_n : A \qquad \Gamma \vdash B \text{ type}}{\Gamma.B \vdash v_{n+1} : A\mathsf{p}} \qquad \frac{\Gamma \vdash A \text{ type} \qquad \Gamma.A \vdash B \text{ type}}{\Gamma \vdash \Pi(A,B) \text{ type}}$$

$$\frac{\Gamma.A \vdash b : B}{\Gamma \vdash \lambda(A,B,b) : \Pi(A,B)} \qquad \frac{\Gamma \vdash c : \Pi(A,B) \qquad \Gamma.A \vdash a : A}{\Gamma \vdash \mathsf{app}(A,B,c,a) : B[a]} \qquad \frac{}{\Gamma \vdash \mathsf{U}_n \text{ type}}$$

$$\frac{\Gamma \vdash A : \mathsf{U}_n}{\Gamma \vdash A \text{ type}} \qquad \frac{\Gamma \vdash A : \mathsf{U}_n \qquad \Gamma.A \vdash B : \mathsf{U}_n}{\Gamma \vdash \Pi(A,B) : \mathsf{U}_n} \qquad \frac{}{\Gamma \vdash \mathsf{U}_m : \mathsf{U}_n}\ (m < n)$$

$$\frac{\Gamma \vdash A : \mathsf{U}_n}{\Gamma \vdash A : \mathsf{U}_p}\ (n < p) \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash A = B}{\Gamma \vdash a : B}$$

We also have the following conversion rules :

$$\frac{\Gamma \vdash a = a' : A \qquad \Gamma \vdash A = B}{\Gamma \vdash \vdash a = a' : B} \qquad \frac{\Gamma \vdash A = B \qquad \Gamma \vdash A = C}{\Gamma \vdash \vdash B = C} \qquad \frac{A \text{ type}}{A = A}$$

$$\frac{\Gamma \vdash a = b : A \qquad \Gamma \vdash a = c : A}{\Gamma \vdash \vdash b = c : A} \qquad \frac{a : A}{a = a : A} \qquad \frac{\Gamma \vdash A = A' \qquad \Gamma.A \vdash B = B'}{\Gamma \vdash \Pi(A,B) = \Pi(A',B')}$$

$$\frac{\Gamma.A \vdash b : B \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathsf{app}(A,B,\lambda(A,B,b),a) = b[a] : B[a]}\ (\beta\text{-rule}) \qquad \frac{f : \Pi\ A\ B}{f = \lambda(A,B,\mathsf{app}(A\mathsf{p}, B\mathsf{p}^+, f\mathsf{p}, v_0) : \Pi(A,B)}\ (\eta\text{-rule})$$

$$\frac{A = A' : \mathsf{U}_n}{A = A'} \qquad \frac{A = A' : \mathsf{U}_n}{A = A' : \mathsf{U}_p}\ (n < p)$$

We also require the rules that makes = a congruence, even if we don't precise them all.

A.4 The system $\mathcal{T}_P$: Russell without annotations

We specify the system $\mathcal{T}_P$, which is the same as the system $\mathcal{T}_R$, but without typing annotations.

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma.A \vdash v_0 : A\mathsf{p}} \qquad \frac{\Gamma \vdash v_n : A \qquad \Gamma \vdash B \text{ type}}{\Gamma.B \vdash v_{n+1} : A\mathsf{p}} \qquad \frac{\Gamma \vdash A \text{ type} \qquad \Gamma.A \vdash B \text{ type}}{\Gamma \vdash \Pi\,A\,B \text{ type}}$$

$$\frac{\Gamma.A \vdash b : B}{\Gamma \vdash \lambda\ A\ b : \Pi\,A\,B} \qquad \frac{\Gamma \vdash c : \Pi\,A\,B \qquad \Gamma.A \vdash a : A}{\Gamma \vdash c\ a : B[a]} \qquad \frac{}{\Gamma \vdash \mathsf{U}_n \text{ type}}$$

$$\frac{\Gamma \vdash A : \mathsf{U}_n}{\Gamma \vdash A \text{ type}} \qquad \frac{\Gamma \vdash A : \mathsf{U}_n \qquad \Gamma.A \vdash B : \mathsf{U}_n}{\Gamma \vdash \Pi\, A\, B : \mathsf{U}_n} \qquad \frac{}{\Gamma \vdash \mathsf{U}_m : \mathsf{U}_n}\ (m < n)$$

$$\frac{\Gamma \vdash A : \mathsf{U}_n}{\Gamma \vdash A : \mathsf{U}_p}\ (n < p) \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash A = B}{\Gamma \vdash a : B}$$

We also have the following conversion rules :

$$\frac{\Gamma \vdash a = a' : A \qquad \Gamma \vdash A = B}{\Gamma \vdash \vdash a = a' : B} \qquad \frac{\Gamma \vdash A = B \qquad \Gamma \vdash A = C}{\Gamma \vdash \vdash B = C} \qquad \frac{A \text{ type}}{A = A}$$

$$\frac{\Gamma \vdash a = b : A \qquad \Gamma \vdash a = c : A}{\Gamma \vdash \vdash b = c : A} \qquad \frac{a : A}{a = a : A} \qquad \frac{\Gamma \vdash A = A' \qquad \Gamma.A \vdash B = B'}{\Gamma \vdash \Pi\, A\, B = \Pi\, A'\, B'}$$

$$\frac{\Gamma.A \vdash b : B \qquad \Gamma \vdash a : A}{\Gamma \vdash ((\lambda\, A\, b)\ a) = b[a] : B[a]}\ (\beta\text{-rule}) \qquad \frac{f : \Pi\, A\, B}{f = \lambda\, A\, (f \mathsf{p}\, v_0) : \Pi\, A\, B}\ (\eta\text{-rule})$$

$$\frac{A = A' : \mathsf{U}_n}{A = A'} \qquad \frac{A = A' : \mathsf{U}_n}{A = A' : \mathsf{U}_p}\ (n < p)$$

B Detailed equivalence proofs

B.1 $\mathcal{T}_T$ and $\mathcal{T}_R$

We give the detailed proof for the two main lemmas :

Proof. (Of lemma 5.12)

We have $\Gamma \vdash A_0 = A_1$, using the rule for transitivity we have $\Gamma \vdash \mathsf{U}_{n_0} = \mathsf{U}_{n_1}$ and necessarily : $n_0 = n_1$, written n.

Then, using $\Gamma \vdash u_0 = t_k^n(v_0) : A_0$ and $\Gamma \vdash u_1 = t_k^n(v_1) : A_1$, we use the rule for type equality that gives us the same derivations with A_0 in both. Moreover, we transform the $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$ derivation to : $\Gamma \vdash t_k^n(v_0) = t_k^n(v_1) : \mathsf{U}_n$. Finally, considering $\Gamma \vdash A_0 = U_n$, we use transitivity to obtain : $\Gamma \vdash u_0 = u_1 : A_0$.

□

Proof. (Of proposition C.5)

We prove the lemma by mutual induction, proving both statements at the same time.

First, let's reason by induction on the lexicographical order based on first the term u_0 and then on the derivation $\Gamma \vdash u_0 : A_0$.

- There is one rule that can always be applied, and justified the choice of the lexicographical order for the induction :

$$\frac{\dfrac{\Pi}{\Gamma \vdash u_0 : A_0'} \qquad \dfrac{\Pi'}{\Gamma \vdash A_0' = A_0}}{\Gamma \vdash u_0 : A_0}$$

  Then, we just have to apply the induction hypothesis on Π, and transitivity with Π' to get the desired result.

- If u_0 is a λ-abstraction, then by inversion, the derivation $\Gamma \vdash u_0 : A_0$ looks like :

$$\dfrac{\dfrac{\Pi_0}{\Gamma, C_0 \vdash b_0 : B_0}}{\Gamma \vdash \lambda(C_0, B_0, b_0) : \Pi(C_0, B_0)}$$

  Then $|u_0| = \lambda(|C_0|, |B_0|, |b_0|) = |u_1|$ and so u_1 is necessarily an abstraction : $u_1 = \lambda(C_1, B_1, b_1)$ with $|C_1| = |C_0|$, $|B_1| = |B_0|$, $|b_1| = |b_0|$. Also, by inversion $\Gamma \vdash u_1 : A_1$ is necessarily :

$$\dfrac{\dfrac{\Pi_1}{\Gamma, C_1 \vdash b_1 : B_1}}{\Gamma \vdash \lambda(C_1; B_1, b_1) : \Pi(C_1, B_1)}$$

  With lemma 5.10 and the first case of the induction hypothesis, we deduce that $\Gamma \vdash C_0 = C_1$. Then, we modify the derivation Π_1 using lemma 5.11 to get a derivation $\Gamma, C_0 \vdash b_1 : B_1$. Thus, we use the induction hypothesis along with lemmas 5.10 and 5.12 to get $\Gamma \vdash b_1 = b_0 : B_0$. Then, with the rule on conversion of product types we finally get : $\Gamma \vdash A_0 = A_1$, and with the one for λ-abstractions we obtain the desired $\Gamma \vdash u_0 = u_1 : A_0$.
- If u_0 is an application, then by inversion, the last rule applied in $\Gamma \vdash u_0 : A_0$ is :

$$\dfrac{\dfrac{\Pi_0}{\Gamma \vdash c_0 : \Pi(C_0, B_0)} \qquad \dfrac{\Pi'_0}{\Gamma \vdash a_0 : C_0}}{\Gamma \vdash \mathsf{app}(C_0, B_0, c_0, a_0,) : B_0[a_0]}$$

  Then $|u_0| = \mathsf{app}(|C_0|, |B_0|, |c_0|, |a_0|) = |u_1|$ and so u_1 is necessarily an application : $u_1 = \mathsf{app}(C_1, B_1, c_1, a_1)$ with $|C_1| = |C_0|$, $|B_1| = |B_0|$, $|c_1| = |c_0|$, $|a_1| = |a_0|$. Also, by inversion : $\Gamma \vdash u_1 : A_1$ is necessarily :

$$\dfrac{\dfrac{\Pi_1}{\Gamma \vdash c_1 : \Pi(C_1, B_1)} \qquad \dfrac{\Pi'_1}{\Gamma \vdash a_1 : C_1}}{\Gamma \vdash \vdash \mathsf{app}(C_1, B_1, c_1, a_1,) : B_1[a_1]}$$

  And so by lemma 5.10 and the induction hypothesis we get a premise $\Gamma \vdash \Pi(C_0, B_0) = \Pi(C_1, B_1)$. Hence with lemma 5.12 and the induction hypothesis, we deduce : $\Gamma \vdash c_0 = c_1 : \Pi(C_0, B_0)$. Also, by using Π-injectivity, we get : $\Gamma \vdash C_0 = C_1$ and $\Gamma \vdash B_0 = B_1$.

  We also get that $\Gamma : a_0 = a_1 : C_0$ with the induction hypothesis and the previous lemma applied on Π'_0 and Π'_1. Finally, with the convertibility rule for app, and the substitution lemma, we get the desired $\Gamma \vdash u_0 = u_1 : A_0$ and $\Gamma \vdash A_0 = A_1$.
- If u_0 is a lifting function, then by inversion we have :

$$\dfrac{\dfrac{\Pi}{\Gamma \vdash a : \mathsf{U}_m}}{\Gamma \vdash t^n_m(a) : U_n}\ (m < n)$$

  Then $|u_0| = |a| = |u_1|$, and with the induction hypothesis on $\Gamma \vdash a : \mathsf{U}_m$ and $\Gamma \vdash u_1 : A_1$, we get two cases.

  <u>First case :</u> We have $\Gamma \vdash \mathsf{U}_m = A_1$ and $\Gamma \vdash a = u_1 : \mathsf{U}_m$. Then immediately we have $\Gamma \vdash A_0 = \mathsf{U}_n$, $\Gamma \vdash A_1 = \mathsf{U}_m$ and $\Gamma \vdash u_0 = t^n_m(a) : A_0$, $\Gamma \vdash u_1 = t^m_m(a) : A_1$ with $\Gamma \vdash a = a : \mathsf{U}_m$ by the reflexivity rule.

  <u>Second case :</u> We have $\Gamma \vdash \mathsf{U}_m = \mathsf{U}_{n_0}$, $\Gamma \vdash A_1 = \mathsf{U}_{n_1}$ and $\Gamma \vdash u_0 = t^{n_0}_k(v_0) : A_0$, $\Gamma \vdash u_1 = t^{n_1}_k(v_1) : A_1$ with $\Gamma \vdash v_0 = v_1 : U_k$. Then necessarily : $n_0 = m$ and we have with the right

rules : $\Gamma \vdash u_0 = t^n_m(t^m_k(v_0)) : U_n$. Hence, with the rule that simplifies lifting operators, $\Gamma \vdash u_0 = t^n_k(v_0) : \mathsf{U}_n$ which is what we wanted as $A_0 = \mathsf{U}_n$.

- If u_0 is a code for a product type, then by inversion :

$$\dfrac{\dfrac{\Pi_0}{\Gamma \vdash a_0 : \mathsf{U}_{n_0}} \qquad \dfrac{\Pi'_0}{\Gamma, T_{n_0}(a_0) \vdash b_0 : \mathsf{U}_{n_0}}}{\Gamma \vdash \Pi^{n_0}\, a_0\, b_0 : \mathsf{U}_{n_0}}$$

Then $|u_0| = \Pi|a_0||b_0| = |u_1|$ and so u_1 is necessarily a code for a dependent product : $u_1 = \Pi^{n_1}\, a_1\, b_1$ with $|a_1| = |a_0|$ and $|b_1| = |b_0|$. Moreover, by inversion, the derivation $\Gamma \vdash u_1 : A_1$ is :

$$\dfrac{\dfrac{\Pi_1}{\Gamma \vdash a_1 : \mathsf{U}_{n_1}} \qquad \dfrac{\Pi'_1}{\Gamma, T_{n_1}(a_1) \vdash b_1 : \mathsf{U}_{n_1}}}{\Gamma \vdash \Pi^{n_1}\, a_1\, b_1 : \mathsf{U}_{n_1}}$$

We have premises $\Gamma \vdash T_{n_0}(a_0)$ type and $\Gamma \vdash T_{n_0}(a_0)$ type because of the lemma 5.10. Hence, we apply the induction hypothesis to get $\Gamma \vdash T_{n_0}(a_0) = T_{n_1}(a_1)$. Hence by lemma 5.11 and the induction hypothesis on Π'_0 and Π'_1, as well as on Π_0 and Π_1, we get three cases.
<u>First case :</u> If we have $\Gamma \vdash \mathsf{U}_{n_0} = \mathsf{U}_{n_1}$ and $\Gamma \vdash b_0 = b_1 : \mathsf{U}_n$, then with the induction hypothesis and lemma 5.12, we easily get $\Gamma \vdash a_0 = a_1$.Thus, we get with the convertibility rule for Pi^n terms, the desired $\Gamma \vdash u_0 = u_1 : \mathsf{U}_{n_0}$.

<u>Second case :</u> If we have $\Gamma \vdash \mathsf{U}_{n_0} = \mathsf{U}_{n_1}$ and $\Gamma \vdash a_0 = a_1 : \mathsf{U}_n$. Then we do an analogous proof as in the first case.

<u>Third case :</u> We have $\Gamma \vdash \mathsf{U}_{n_0} = \mathsf{U}_{n'_0}$ and $\Gamma \vdash \mathsf{U}_{n_1} = \mathsf{U}_{n'_1}$, $\Gamma \vdash b_0 = t^{n_0}_k(c_0)$ and $\Gamma \vdash b_1 = t^{n_1}_k(c_1)$, with $\Gamma \vdash c_0 = c_1 : \mathsf{U}_k$. And also : $\Gamma \vdash a_0 = t^{n_0}_l(d_0)$ and $\Gamma \vdash a_1 = t^{n_1}_l(d_1)$, with $\Gamma \vdash d_0 = d_1 : \mathsf{U}_k$. Then, necessarily $n'_0 = n_0$ and $n'_1 = n_1$. We take $m = \max(k, l)$, and construct derivations $\Gamma \vdash b_0 = t^{n_0}_m(t^m_k(c_0))$ and $\Gamma \vdash b_1 = t^{n_1}_m(t^m_k(c_1))$, with $\Gamma \vdash t^m_k(c_0) = t^m_k(c_1) : \mathsf{U}_m$. And also : $\Gamma \vdash a_0 = t^{n_0}_m(t^m_l(d_0))$ and $\Gamma \vdash a_1 = t^{n_1}_m(t^m_l(d_1))$, with $\Gamma \vdash t^m_l(d_0) = t^m_l(d_1) : \mathsf{U}_k$. Finally, we use the rule which propagates coercions along Π^{n_1} and Π^{n_0} to get the second case of our lemma.

- If u_0 is a code for a universe, then by inversion :

$$\dfrac{}{\Gamma \vdash \mathsf{U}^{n_0}_m : \mathsf{U}_{n_0}}$$

Then $|u_0| = \mathsf{U}_m = |u_1|$ so $u_1 = \mathsf{U}^{n_1}_m$ or $u_1 = t^{n_1}_l(\mathsf{U}^l_m)$.

<u>First case :</u> We have $u_1 = \mathsf{U}^{n_1}_m$. Then we define $k := \min(n_0, n_1)$ and we have the following derivations : $\Gamma \vdash A_0 = \mathsf{U}_{n_0}$ and $\Gamma \vdash A_1 = \mathsf{U}_{n_1}$, along with $\Gamma \vdash u_0 = t^{n_0}_k(\mathsf{U}^k_m)$ and $\Gamma \vdash u_1 = t^{n_1}_k(\mathsf{U}^k_m)$, with $\Gamma \vdash \mathsf{U}^k_m = \mathsf{U}^k_m : \mathsf{U}_k$.

<u>Second case :</u> We have $u_1 = t^{n_1}_l(\mathsf{U}^l_m)$, and we define $k := \min(l, n_0)$. So that we have the following derivations : $\Gamma \vdash A_0 = \mathsf{U}_{n_0}$ and $\Gamma \vdash A_1 = \mathsf{U}_{n_1}$, along with $\Gamma \vdash u_0 = t^{n_0}_k(\mathsf{U}^k_m)$ and $\Gamma \vdash u_1 = t^{n_1}_k(\mathsf{U}^k_m)$, with $\Gamma \vdash \mathsf{U}^k_m = \mathsf{U}^k_m : \mathsf{U}_k$.

- If u_0 is v_0, then by inversion :

$$\dfrac{\dfrac{\Pi_0}{\Gamma \vdash B_0 \text{ type}}}{\Gamma, B_0 \vdash v_0 : B_0 \mathsf{p}}$$

then $|u_0| = v_0 = |u_1|$ and so necessarily $u_1 = v_0$. Reasoning by inversion, it is necessarily the same rule which is applied last in the derivation $\Gamma \vdash u_1 : A_1$. Hence, because the two contexts are the same, we have $A_1 = B_0\mathsf{p}$ and hence we get trivially : $\Gamma \vdash u_0 = u_1 : A_0$ and $\Gamma \vdash A_0 = A_1$.

- If u_0 is a non v_0 variable, then by inversion :

$$\frac{\Gamma \vdash v_i : C_0 \qquad \Gamma \vdash B_0 \text{ type}}{\Gamma, B_0 \vdash v_{i+1} : C_0\mathsf{p}}$$

Then $|u_0| = v_{i+1} = |u_1|$, and necessarily $u_1 = v_{i+1}$. Reasoning by inversion on $\Gamma \vdash u_1 : A_1$, the last rule applied should be :

$$\frac{\Gamma \vdash v_i : C_1 \qquad \Gamma \vdash B_0 \text{ type}}{\Gamma, B_0 \vdash v_{i+1} : C_1\mathsf{p}}$$

But, reasoning by inversion on the derivation $\Gamma \vdash v_i : C_1$, it is also a variable rule which is applied, and hence $\Gamma = \Gamma', C_1$. Thus, we necessarily have $C_0 = C_1$ because the contexts are the same. Hence, we get trivially $\Gamma \vdash u_0 = u_1 : A_0$ and $\Gamma \vdash A_0 = A_1$.

All of this concludes the proof for the second part of the statement, but we also need to prove the first part of the statement as part of the mutual induction. So let's reason on the type A.

- If A is a Π-type, then by inversion the derivation $\Gamma \vdash A$ type has the form :

$$\frac{\Gamma \vdash A_1 \text{ type} \qquad \Gamma.A_1 \vdash A_2 \text{ type}}{\Gamma \vdash \Pi\, A_1\, A_2 \;\; \text{type}}$$

Then $|A| = \Pi(|A_1|, |A_2|) = |B|$, and B should be a Π type : $B = \Pi(B_1, B_2)$ with $|B_1| = |A_1|$ and $|B_2| = |A_2|$. So, by the induction hypothesis we get : $\Gamma \vdash A_1 = B_1$. Then, with the lemma 5.11 and the induction hypothesis, we also have the derivation $\Gamma, A_1 \vdash A_2 = B_2$. Hence with the rule on convertibility of Π types, we get the desired : $\Gamma \vdash \Pi(A_1, A_2) = \Pi(B_1, B_2)$.

- If A is a decoded type $T_n(a)$, then by inversion :

$$\frac{\dfrac{\Pi}{\Gamma \vdash a : \mathsf{U}_n}}{\Gamma \vdash T_n(a) \text{ type}}$$

Then $|A| = |a| = |B|$ and we reason by case on the last rule of $\Gamma \vdash B$ type, and there is only two cases as the erasure of B should be the erasure of a term :

<u>First case :</u> The last rule was :

$$\frac{\Gamma \vdash B_1 \text{ type} \qquad \Gamma.B_1 \vdash B_2 \text{ type}}{\Gamma \vdash \Pi\, B_1\, B_2 \;\; \text{type}}$$

Then $|B| = \Pi(|B_1|, |B_2|) = |a|$, and so $a = \Pi^n\, a_1\, a_2$ necessarily, with $|a_1| = |B_1|$ and $|a_2| = |B_2|$. Hence the derivation $\Gamma \vdash a : \mathsf{U}_n$ is :

$$\frac{\dfrac{\Pi_1}{\Gamma \vdash a_1 : \mathsf{U}_n} \qquad \dfrac{\Pi_2'}{\Gamma, T_n(a_1) \vdash a_2 : \mathsf{U}_n}}{\Gamma \vdash \Pi^n\, a_1\, a_2 : \mathsf{U}_n}$$

And so by the lemma 5.10, we have premises $\Gamma \vdash T_n(a_1)$ type and $\Gamma, T_n(a_1) \vdash T_n(a_2)$ type. So by the induction hypothesis and the lemma 5.11 we get : $\Gamma \vdash T_n(a_1) = B_1$ and $\Gamma, T_n(a_1) \vdash T_n(a_2) = B_2$.
We also have : $\Gamma \vdash T_n(\Pi^n\, a_1\, a_2) = \Pi(T_n(a_1), T_n(a_2))$, and hence we get $\Gamma \vdash T_n(a) = \Pi(B_1, B_2)$ which is the desired $\Gamma \vdash A = B$.

<u>Second case :</u> If the last rule was :

$$\dfrac{\dfrac{\Pi'}{\Gamma \vdash b : \mathsf{U}_m}}{\Gamma \vdash T_m(b)\ \mathsf{type}}$$

Then $|A| = |a| = |B| = |b|$ and so by the induction hypothesis on Π and Π', we get (by the second statement) two cases :

- If $\Gamma \vdash \mathsf{U}_n = \mathsf{U}_m$ and $\Gamma \vdash a = b : \mathsf{U}_n$. Then $n = m$ by injectivity of universes and so : $\Gamma \vdash T_n(a) = T_n(b)$ i.e $\Gamma \vdash A = B$.
- If $\Gamma \vdash \mathsf{U}_n = \mathsf{U}_{n_0}$, $\Gamma \vdash \mathsf{U}_m = \mathsf{U}_{n_1}$ and $\Gamma \vdash a = t_k^{n_0}(v_0) : \mathsf{U}_{n_0}$, $\Gamma \vdash b = t_k^{n_1}(v_1) : \mathsf{U}_{n_1}$ with $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$.
  Then, we have necessarily $n = n_0$, $m = n_1$ and deduce that $\Gamma \vdash T_n(a) = T_n(t_k^n(v_0))$, and $\Gamma \vdash T_m(b) = T_n(t_k^m(v_1))$ which can be simplified as $\Gamma \vdash A = T_k(v_0)$, and $\Gamma \vdash B = T_k(v_1)$. Finally, by using $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$ we get the desired $\Gamma \vdash A = B$.

- If A is a universe, then by inversion :

$$\dfrac{}{\Gamma \vdash \mathsf{U}_n\ \mathsf{type}}$$

Then $|A| = \mathsf{U}_n = |B|$ and so necessarily $B = \mathsf{U}_n$ or $B = T_k(\mathsf{U}_n^k)$, and either way we have a derivation for $\Gamma \vdash A = B$.

All of this concludes this proof. □

Then, we prove the main equivalence result :

Proof. (Of theorem 5.15)

We prove existence and uniqueness of the derivation in $\mathcal{T}_T$.

Uniqueness : (up to conversion)
Let $\Gamma \vdash J$ be a judgment in the system $\mathcal{T}_R$, and assume that there exists $\Gamma_1 \vdash J_1$ and $\Gamma_1' \vdash J_1'$ in $\mathcal{T}_T$ so that : $|\Gamma_1| = \Gamma = |\Gamma_1'|$ and $|J_1| = J = |J_1'|$.

Then, we directly have with the previous corollary (depending on the kind of judgment), that $\Gamma_1 \vdash J_1 = J_2$.

Existence : We show that we can lift the judgment by induction on the derivation $\Gamma \vdash J$ (in $\mathcal{T}_R$).

- If the last rule applied is :

$$\dfrac{\dfrac{\Pi}{\Gamma \vdash A : \mathsf{U}_n}}{\Gamma \vdash A\ \mathsf{type}}$$

Then we construct a derivation $\Gamma' \vdash a : B'$ in $\mathcal{T}_T$ by induction hypothesis, such that $|\Gamma' \vdash a : B'| = \Gamma \vdash A : \mathsf{U}_n$. Hence, as B' is a type such that $|B'| = \mathsf{U}_n$, we have $B' = \mathsf{U}_n$ necessarily. Then, we construct the following derivation :

$$\dfrac{\dfrac{\Pi'}{\Gamma' \vdash a : \mathsf{U}_n}}{\Gamma' \vdash T_n(a)\ \mathsf{type}}$$

And we have the desired : $|\Gamma' \vdash T_n(a)\ \mathsf{type}| = \Gamma \vdash A\ \mathsf{type}$ because $|T_n(a)| = |a| = A$.

- If the last rule applied is :

$$\dfrac{\dfrac{\Pi_1}{\Gamma \vdash A\,\mathsf{type}} \qquad \dfrac{\Pi_2}{\Gamma, A \vdash B\,\mathsf{type}}}{\Gamma \vdash \Pi(A, B)\ \mathsf{type}}$$

Then, we get by the induction hypothesis that there exists derivable judgments $\Gamma' \vdash A'$ type and $\Gamma'', A'' \vdash B'$ type such that the erasure correspond to Π_1 and Π_2. We have $|\Gamma'| = |\Gamma''| = \Gamma$ and $|A''| = |A'| = A$, and thus by uniqueness up to conversion (with the previous lemma), the contexts Γ' and Γ'' are convertible. Thus, we transform the second derivation to get $\Gamma', A' \vdash B'$ type. And hence, we construct the derivation in $\mathcal{T}_T$:

$$\dfrac{\dfrac{\Pi_1'}{\Gamma' \vdash A' \text{ type}} \qquad \dfrac{\Pi_2'}{\Gamma', A' \vdash B' \text{ type}}}{\Gamma' \vdash \Pi(A', B') \text{ type}}$$

- If the last rule applied is :

$$\dfrac{\dfrac{\Pi_1}{\Gamma \vdash A : \mathsf{U}_n} \qquad \dfrac{\Pi_2}{\Gamma, A \vdash B : \mathsf{U}_n}}{\Gamma \vdash \Pi(A, B) : \mathsf{U}_n}$$

Then, we get by the induction hypothesis that there exists derivable judgments $\Gamma' \vdash a : U$ and $\Gamma'', A' \vdash b : U'$ such that the erasure correspond to Π_1 and Π_2. We necessarily have $U = U' = \mathsf{U}_n$ by the same argument as before. Moreover, we have $|\Gamma'| = |\Gamma''| = \Gamma$, and thus by uniqueness up to conversion (with the previous lemma), the contexts Γ and Γ' are convertible. Thus, we transform the second derivation to get $\Gamma', A' \vdash b : \mathsf{U}_n$. Also, the equality $|A'| = |a| = A$ gives us two cases to consider:

<u>First case :</u> We have $A' = T_n(a)$. Hence, we immediately apply the rule for Π^n formation :

$$\dfrac{\dfrac{\Pi_1'}{\Gamma' \vdash a : \mathsf{U}_n} \qquad \dfrac{\Pi_2'}{\Gamma', T_n(a) \vdash b : \mathsf{U}_n}}{\Gamma' \vdash \Pi^n\ a\ b\ : \mathsf{U}_n}$$

<u>Second case :</u> We have $A' = \mathsf{U}_m$ and $a = \mathsf{U}_m^n$, then we use the rule $\Gamma' \vdash T_n(\mathsf{U}_m^n) = \mathsf{U}_m$ which gives us $\Gamma' \vdash T_n(a) = A'$, which allows us to transform the derivation $\Gamma', A' \vdash b : \mathsf{U}_n$ and do the same thing as in the first case.

- If the last rule applied is :

$$\dfrac{}{\Gamma \vdash \mathsf{U}_m : \mathsf{U}_n}\ (m < n)$$

Then we produce the following derivation :

$$\dfrac{}{\Gamma \vdash \mathsf{U}_m^n : \mathsf{U}_n}\ (m < n)$$

- If the last rule applied is :

$$\dfrac{\dfrac{\Pi}{\Gamma \vdash A : \mathsf{U}_n}}{\Gamma \vdash A : \mathsf{U}_p}\ (n < p)$$

Then we use the induction hypothesis on Π, which gives us a derivation $\Gamma' \vdash a : \mathsf{U}_n$ such that its erasure is $\Gamma \vdash A : \mathsf{U}_n$. Hence, we produce the following derivation, which gives us the desired result.

$$\dfrac{\dfrac{\Pi}{\Gamma' \vdash a : \mathsf{U}_n}}{\Gamma' \vdash t_n^p(a) : \mathsf{U}_p}\ (n < p)$$

- If the last rule is :

$$\dfrac{\dfrac{\Pi}{\Gamma' \vdash A = A' : \mathsf{U}_n}}{\Gamma' \vdash A = A'}$$

By the induction hypothesis, we get a derivation $\Gamma' \vdash a = a' : \mathsf{U}_n$, and we produce :

$$\dfrac{\dfrac{\Pi}{\Gamma' \vdash a = a' : \mathsf{U}_n}}{\Gamma' \vdash T_n(a) = T_n(a')}$$

- If the last rule is :

$$\dfrac{\dfrac{\Pi}{\Gamma' \vdash A = A' : \mathsf{U}_n}}{\Gamma' \vdash A = A' : \mathsf{U}_p}\,(n < p)$$

By the induction hypothesis, we get a derivation $\Gamma' \vdash a = a' : \mathsf{U}_n$, and we produce :

$$\dfrac{\dfrac{\Pi}{\Gamma' \vdash a = a' : \mathsf{U}_n}}{\Gamma' \vdash t_n^p(a) = t_n^p(a') : \mathsf{U}_p}\,(n < p)$$

- If the rule has a direct corresponding rule in the system $\mathcal{T}_T$, we have a direct proof by using the induction hypothesis and invoking uniqueness up to conversion (for rules with two premises).

□

B.2 $\mathcal{T}_R$ and $\mathcal{T}_P$

We give the detailed proof for the main results :

Proof. (Of lemma 5.18)

Here, again we prove the two statements simultaneously, by mutual induction.

First, by induction on the lexicographical order formed by the term u_0 and its derivation :

- If u_0 is a product, the the last rule applied is the product rule :

$$\dfrac{\dfrac{\Pi_1}{\Gamma \vdash C : \mathsf{U}_n} \qquad \dfrac{\Pi_2}{\Gamma, C \vdash D : \mathsf{U}_n}}{\Gamma \vdash \Pi(C, D) : \mathsf{U}_n}$$

Then we have $\mathsf{strip}(u_0) = \Pi\,(x : \mathsf{strip}(C))\,\mathsf{strip}(D) = \mathsf{strip}(u_1)$, and hence u_1 is a product type : $u_1 = \Pi(C', D')$ with the analogous derivation. Thus, we apply the induction hypothesis on the premises, with modifying the context of the second one as usual, and we get : $\Gamma' \vdash C = C' : \mathsf{U}_n$ and $\Gamma', C' \vdash D = D' : \mathsf{U}_n$. Finally, we construct a derivation $\Gamma' \vdash \Pi(C, D) = \Pi(C', D') : \mathsf{U}_n$ in $\mathcal{T}_R$.
- If $u_0 = \mathsf{U}_n$, then $u_1 = \mathsf{U}_n$ necessarily, and we trivially have $\Gamma \vdash A = B$ in $\mathcal{T}_R$.
- If u_0 is an abstraction, then the last rule applied is :

$$\dfrac{\dfrac{\Pi_0}{\Gamma, C_0 \vdash b_0 : B_0}}{\Gamma \vdash \lambda(C_0; B_0, b_0) : \Pi(C_0, B_0)}$$

Then, $\mathsf{strip}(u_0) = \lambda\,(x : \mathsf{strip}(C_0))\,\mathsf{strip}(b_0) = \mathsf{strip}(u_1)$, and so we have $u_1 = \lambda(C_1, B_1, b_1)$ with $\mathsf{strip}(C_0) = \mathsf{strip}(C_1)$ and $\mathsf{strip}(b_0) = \mathsf{strip}(b_1)$, with the analogous derivation Π_1.

Hence, with an adapted lemma 5.10 for $\mathcal{T}_R$ to get premises $\Gamma \vdash C_0$ type and $\Gamma \vdash C_1$ type that gives us with the second statement of the induction hypothesis : $\Gamma \vdash C_0 = C_1$. and thus we transform Π_1 to a derivation $\Gamma, C_0 \vdash b_1 : B_1$. By applying the induction hypothesis, we get :

$\Gamma \vdash b_0 = b_1 : B_0$ and $\Gamma \vdash B_0 = B_1$. Finally, we construct : $\Gamma \vdash u_0 = u_1 : A_0$ and $\Gamma \vdash A_0 = A_1$.

Note that here we didn't have the hypothesis $\mathsf{strip}(B_0) = \mathsf{strip}(B_1)$.

- The case of application is proved in a similar way, with the help of Π-injectivity.
- The case where u_0 is a universe, and so the last rule applied is :

$$\frac{}{\Gamma \vdash \mathsf{U}_m : \mathsf{U}_n}\ (m < n)$$

This case explains why didn't ask for $\Gamma \vdash A_0 = A_1$ in the consequences of the first case in our lemma. In fact, here we just have syntactically $u_1 = \mathsf{U}_m$ as they have the same stripping, so we directly have $\Gamma \vdash u_0 = u_1 : \mathsf{U}_m$ but we don't necessarily have A_0 and A_1 convertible, because they can be different universes.

Then, by induction on A.

- If A is a product type and the the last rule applied is the product rule :

$$\frac{\dfrac{\Pi_1}{\Gamma \vdash C\ \mathsf{type}} \qquad \dfrac{\Pi_2}{\Gamma, C \vdash D\ \mathsf{type}}}{\Gamma \vdash \Pi(C, D)\ \mathsf{type}}$$

Then we the same proof as in the first induction, but with type judgments.

- If the last rule applied is :

$$\frac{\Gamma \vdash A : \mathsf{U}_n}{\Gamma \vdash A\ \mathsf{type}}$$

Then, we apply the induction hypothesis for the first statement, and get $\Gamma \vdash A = B : \mathsf{U}_n$, with which we construct : $\Gamma \vdash A = A'$.

- We don't detail the other cases.

□

Proof. (Of theorem 5.19)

By induction on the derivation $\Gamma \vdash J$, with the previous lemma, we construct a derivation in $\mathcal{T}_R$. We only detail the interesting cases, which are abstraction and application.

- If the last rule applied is :

$$\frac{\dfrac{\Pi_1}{\Gamma \vdash c : \Pi(A, B)} \qquad \dfrac{\Pi_2}{\Gamma \vdash a : A}}{\Gamma \vdash c\ a : B[a]}$$

Then, by induction on Π_1, we get a derivation in $\mathcal{T}_R$ of $\Gamma' \vdash c' : T$ such that $\mathsf{strip}(\Gamma') = \Gamma$, $\mathsf{strip}(c') = c$ and $\mathsf{strip}(T) = \Pi(A, B)$. Hence, necessarily T is a product type $\Pi(A', B')$ such that $\mathsf{strip}(A') = A$ and $\mathsf{strip}(B') = B$.
By induction on Π_2, we also get a derivation in $\mathcal{T}_R$ of $\Gamma'' \vdash a' : A''$ with the desired strippings. Hence, because of the previous lemma, and given $\mathsf{strip}(\Gamma') = \mathsf{strip}(\Gamma'')$, the two contexts are necessarily convertible. In the same way, with the previous lemma we get a derivation : $\Gamma' \vdash A' = A''$, and we thus transform the derivation to get $\Gamma' \vdash a' : A'$. Finally, we construct :

$$\frac{\dfrac{\Pi_1}{\Gamma' \vdash c' : \Pi(A', B')} \qquad \dfrac{\Pi_2}{\Gamma' \vdash a' : A'}}{\Gamma \vdash \mathsf{app}(A', B', c', a') : B'[a']}$$

Which satisfies the desired property.

- If the last rule applied is :

$$\dfrac{\dfrac{\Pi}{\Gamma, A \vdash b : B}}{\Gamma \vdash \lambda(x : A)b : \Pi(A, B)}$$

Then, by induction on Π, we get a derivation in $\mathcal{T}_R$ of $\Gamma', A' \vdash b' : B'$ such that $\mathrm{strip}(\Gamma') = \Gamma$, $\mathrm{strip}(b') = b$, $\mathrm{strip}(A') = A$ and $\mathrm{strip}(B') = B$.
Hence, we construct the derivation :

$$\dfrac{\dfrac{\Pi'}{\Gamma', A' \vdash b' : B'}}{\Gamma' \vdash \lambda(A', B', b') : \Pi(A', B')}$$

Which satisfies the desired property.

- We do not detail the other cases, that can be treated similarly.

□

C Adding Universe Polymorphism

C.1 The Russell/Tarski equivalence

We consider the system described in [3], with cumulative Tarski universes and level constraints. This system can be seen as an extension of the system $\mathcal{T}_T$ and we will refer to it as $\mathcal{T}_T^*$, while the analogous theory with Russell universes (also described in [3]) will be referred to as $\mathcal{T}_R^*$.

Let's extend the erasure operation :

Definition C.1 (Erasure). We define the erasure operation, denoted $|.|$:

- $|\mathsf{U}_l| = \mathsf{U}_l$.
- $|\mathsf{U}_l^m| = \mathsf{U}_m$.
- $|\mathrm{app}(A, B, c, a)| = \mathrm{app}(|A|, |B|, |c|, |a|)$.
- $|\lambda(A, B, b)| = \lambda(|A|, |B|, |b|)$.
- $| < \alpha > a| =< \alpha > |a|$
- $| < \psi > a| =< \psi > |a|$
- $|v_i| = v_i$.
- $|T_l(a)| = |a|$.
- $|t_l^m(a)| = |a|$.
- $|\Pi(A, B)| = \Pi(|A|, |B|)$.
- $|\Pi^l\, a\, b| = \Pi\ |a|\ |b|$.
- $|[\psi]A| = [\psi]|A|$

We show that this operation indeed preserves derivations.

Lemma C.2. *If $\Gamma \vdash J$ in $\mathcal{T}_T^*$, then $|\Gamma| \vdash |J|$ in $\mathcal{T}_R^*$.*

Proof. By direct induction. □

We prove the technical lemmas like in subsection 5.2, but we do not give details for that. Let's prove one more technical lemma :

Lemma C.3. *If we have a derivation for a code $\Gamma \vdash \mathsf{U}_l^m$ or $\Gamma \vdash t_l^m(a)$ in $\mathcal{T}_T^*$ then we have a derivation $\Gamma \vdash l \leq m$.*

Proof. By inversion, we find directly the derivation in the rules. □

Lemma C.4. *Let's assume that : $\Gamma \vdash A_0 = \mathsf{U}_{l_0}$, $\Gamma \vdash A_1 = \mathsf{U}_{l_1}$ and $\Gamma \vdash u_0 = t_k^{l_0}(v_0) : A_0$, $\Gamma \vdash u_1 = t_k^{l_1}(v_1) : A_1$ with $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$.*

Then if $\Gamma \vdash A_0 = A_1$, we have : $\Gamma \vdash u_0 = u_1 : A_0$.

Proof. Here, injectivity of universes gives us equality in the semi-lattice. We have $\Gamma \vdash A_0 = A_1$, using the rule for transitivity we have $\Gamma \vdash \mathsf{U}_{l_0} = \mathsf{U}_{l_1}$ and by injectivity for universes : $l_0 = l_1$ written

n in the sup semi-lattice. Then, we conduct exactly the same proof.

Using $\Gamma \vdash u_0 = t_k^n(v_0) : A_0$ and $\Gamma \vdash u_1 = t_k^n(v_1) : A_1$, we use the rule for type equality that gives us the same derivations with A_0 in both. Moreover, we transform the $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$ derivation to : $\Gamma \vdash t_k^n(v_0) = t_k^n(v_1) : \mathsf{U}_n$, as because of lemma C.3 we have a derivation $\Gamma \vdash k \leq n$. □

Lemma C.5. *If* $\Gamma \vdash A$ type, $\Gamma \vdash B$ type *in* $\mathcal{T}_T$ *and* $|A| = |B|$, *then* $\Gamma \vdash A = B$.
If $\Gamma \vdash u_0 : A_0$ *and* $\Gamma \vdash u_1 : A_1$ *in* $\mathcal{T}_T$ *and* $|u_0| = |u_1|$ *then*

- *Either* $\Gamma \vdash A_0 = A_1$ *and* $\Gamma \vdash u_0 = u_1 : A_0$.
- *Or* $\Gamma \vdash A_0 = \mathsf{U}_{l_0}$, $\Gamma \vdash A_1 = \mathsf{U}_{l_1}$ *and* $\Gamma \vdash u_0 = t_k^{l_0}(v_0) : A_0$, $\Gamma \vdash u_1 = t_k^{l_1}(v_1) : A_1$ *with* $\Gamma \vdash v_0 = v_1 : \mathsf{U}_k$.

Proof. As before, by mutual induction with respect to the lexicographical order on the term u_0 and then on the derivation $\Gamma \vdash u_0 : A_0$. We will consider only some cases to show that the proof extends to universe polymorphism. The proof is basically the same, and we just need to make sure that we have the hypothesis $\Gamma \vdash l \leq m$ - provided by lemma C.3 - to construct our derivations.

- For variables, abstractions and applications, the proof is exactly the same.
- If u_0 is a lifting function, then by inversion we have :

$$\dfrac{\dfrac{\Pi}{\Gamma \vdash a : \mathsf{U}_l} \qquad \dfrac{}{\Gamma \vdash l \leq m}}{\Gamma \vdash t_l^m(a) : U_m}$$

  Then $|u_0| = |a| = |u_1|$, and with the induction hypothesis on $\Gamma \vdash a : \mathsf{U}_l$ and $\Gamma \vdash u_1 : A_1$, we get two cases.

  <u>First case :</u> We have $\Gamma \vdash \mathsf{U}_l = A_1$ and $\Gamma \vdash a = u_1 : \mathsf{U}_l$. Then immediately we have $\Gamma \vdash A_0 = \mathsf{U}_m$, $\Gamma \vdash A_1 = \mathsf{U}_l$ and $\Gamma \vdash u_0 = t_l^m(a) : A_0$, $\Gamma \vdash u_1 = t_l^l(a) : A_1$ (because we have $l \leq m$) with $\Gamma \vdash a = a : \mathsf{U}_l$ by the reflexivity rule.

  <u>Second case :</u> We have $\Gamma \vdash \mathsf{U}_l = \mathsf{U}_{l_0}$, $\Gamma \vdash A_1 = \mathsf{U}_{l_1}$ and $\Gamma \vdash u_0 = t_k^{l_0}(v_0) : A_0$, $\Gamma \vdash u_1 = t_k^{l_1}(v_1) : A_1$ with $\Gamma \vdash v_0 = v_1 : U_k$. Then necessarily : $l_0 = l$ and we have with the right rules : $\Gamma \vdash u_0 = t_l^m(t_k^l(v_0)) : U_m$ which is possible because $k \leq l \leq m$. Hence, with the rule that simplifies lifting operators, $\Gamma \vdash u_0 = t_k^m(v_0) : \mathsf{U}_m$ which is what we wanted as $A_0 = \mathsf{U}_m$.
- If u_0 is a code for a universe, then by inversion :

$$\dfrac{}{\Gamma \vdash \mathsf{U}_m^{l_0} : \mathsf{U}_{l_0}}$$

  Then $|u_0| = \mathsf{U}_m = |u_1|$ so $u_1 = \mathsf{U}_m^{l_1}$ or $u_1 = t_l^{l_1}(\mathsf{U}_m^l)$.

  <u>First case :</u> We have $u_1 = \mathsf{U}_m^{l_1}$. Then we define $k := \min(l_0, l_1)$ informally, which corresponds to defining k to l_0 or l_1 depending on if $l_0 < l_1$ or not. Then, we construct the following derivations : $\Gamma \vdash A_0 = \mathsf{U}_{l_0}$ and $\Gamma \vdash A_1 = \mathsf{U}_{l_1}$, along with $\Gamma \vdash u_0 = t_k^{l_0}(\mathsf{U}_m^k)$ and $\Gamma \vdash u_1 = t_k^{l_1}(\mathsf{U}_m^k)$, with $\Gamma \vdash \mathsf{U}_m^k = \mathsf{U}_m^k : \mathsf{U}_k$.

  <u>Second case :</u> We have $u_1 = t_l^{l_1}(\mathsf{U}_m^l)$, and we define $k := \min(l, l_0)$ (also informally as before). So that we have the following derivations : $\Gamma \vdash A_0 = \mathsf{U}_{l_0}$ and $\Gamma \vdash A_1 = \mathsf{U}_{l_1}$, along with $\Gamma \vdash u_0 = t_k^{l_0}(\mathsf{U}_m^k)$ and $\Gamma \vdash u_1 = t_k^{l_1}(\mathsf{U}_m^k)$, with $\Gamma \vdash \mathsf{U}_m^k = \mathsf{U}_m^k : \mathsf{U}_k$.

- If u_0 is a code for a product type, we do the same proof, adapting it for universe levels by providing the inequality hypothesis. The only difference is in the third case, were we used the max function, which is replaced by $\vee$.
- If u_0 is a constrained term or a level abstraction $[\alpha]t$ or $[\psi]t$, the erasure and inversion allow to use the induction hypothesis directly on t, and we get the result easily.

Now, we prove the second item of the lemma by induction on the type A.

- The proofs for the regular types are exactly the same.
- If A is a level product or a constrained type, then we directly use the induction hypothesis because of the erasure and the typing premise that we get by inversion.

All of this concludes this proof. □

Then, we prove the equivalence result :

Theorem C.6. *If $\Gamma \vdash J$ in the system $\mathcal{T}_R^*$, then there exists a unique, up to conversion judgment $\Gamma_1 \vdash J_1$ in the system $\mathcal{T}_T^*$ such that $|\Gamma_1| = \Gamma$ and $|J_1| = J$.*

Proof. Follows from the previous lemma as before, we specify the rules which needs level inequality hypothesis. The rules for levels or constraints are the same in $\mathcal{T}_T^*$ and $\mathcal{T}_R^*$ so the result is obtained immediately.

- For many rules, the proof is exactly the same as before.
- If the last rule applied is :

$$\frac{\Gamma \vdash m < n}{\Gamma \vdash \mathsf{U}_m : \mathsf{U}_n}$$

  Then we produce the following derivation :

$$\frac{\Gamma \vdash m < n}{\Gamma \vdash \mathsf{U}_m^n : \mathsf{U}_n}$$

- If the last rule applied is :

$$\frac{\dfrac{\Pi}{\Gamma \vdash A : \mathsf{U}_n} \qquad \dfrac{}{\Gamma \vdash n < p}}{\Gamma \vdash A : \mathsf{U}_p}$$

  Then we use the induction hypothesis on Π, which gives us a derivation $\Gamma' \vdash a : \mathsf{U}_n$ such that its erasure is $\Gamma \vdash A : \mathsf{U}_n$. Hence, we produce the following derivation, which gives us the desired result.

$$\frac{\dfrac{\Pi}{\Gamma' \vdash a : \mathsf{U}_n} \qquad \dfrac{}{\Gamma \vdash n < p}}{\Gamma' \vdash t_n^p(a) : \mathsf{U}_p}$$

- If the last rule is :

$$\frac{\dfrac{\Pi}{\Gamma' \vdash A = A' : \mathsf{U}_n} \qquad \dfrac{}{\Gamma \vdash n < p}}{\Gamma' \vdash A = A' : \mathsf{U}_p}$$

  By the induction hypothesis, we get a derivation $\Gamma' \vdash a = a' : \mathsf{U}_n$, and we produce :

$$\frac{\dfrac{\Pi}{\Gamma' \vdash a = a' : \mathsf{U}_n} \qquad \dfrac{}{\Gamma \vdash n < p}}{\Gamma' \vdash t_n^p(a) = t_n^p(a') : \mathsf{U}_p}$$

- If the rule has a direct corresponding rule in the system $\mathcal{T}_T^*$, we have a direct proof by using the induction hypothesis and invoking uniqueness up to conversion (for rules with two premises). This is true in particular for rules on levels and constraints which do not differ between the Russell and Tarski presentations.

□

Finally, we extended our result for a theory with universe polymorphism : the two theories $\mathcal{T}_T^*$ and $\mathcal{T}_R^*$ are equivalent. This means that the Russell presentation is also initial to the notion of model for universe polymorphic theories. This notion of model will be specified in a forthcoming paper and we believe that proving normalisation by adapting the gluing technique should be possible.